

Universidad ORT Uruguay

Facultad de Ingeniería

Diseño de Aplicaciones 2

Entrega - Obligatorio 1

2026

Descripción del diseño

Repositorio

Juan Ferreiro - 328 344

Sebastián Luciardi - 274 754

Andrés Bustamante - 302 420

Grupo M6C - Tutores: Alexander Wieler y Franco Bonavita

Entregado como requisito para la obtención del título de Ingeniero en Sistemas

Índice:

Descripción general de la solución 5
FarmeORT y alcance de la primera entrega 5
Estado de la solución, errores conocidos y supuestos 5
Arquitectura y organización en paquetes 6
Organización según el modelo 4+1 6
Diagrama general de paquetes 6
Organización por capas y dependencias 6
Responsabilidades de cada paquete 7
Vista lógica — clases y relaciones 9
Diagramas de clases por paquete 9
Jerarquías de herencia relevantes 10
Modelo de datos y persistencia 12
Modelo de tablas de la base de datos 12
Persistencia con Entity Framework Core 12
Integridad y consistencia 12
Consistencia, atomicidad y concurrencia en persistencia 13
Vista de procesos — interacciones relevantes 14
Criterio de selección 14
Inscripción a un torneo 15
Lanzamiento de una skill 15
Fallo de batalla y actualización de ELO 16
Manejo de excepciones 17
Inicio de Sesión 17
Justificación del diseño 19
Principios SOLID 19
GRASP 20
Inyección de dependencias 20
Fábrica y composición 21
Patrones de diseño 21
Strategy para el cálculo de ELO 21
Strategy para el emparejamiento 22
Template Method en autenticación y autorización 22
Acceso a datos 23

Manejo de excepciones 23
Impacto sobre la mantenibilidad 23
Decisiones propias, mecanismos y algoritmos relevantes 24
Vista de implementación 25
Estructura de assemblies 25
Diagrama de componentes 26
Dependencias entre componentes 26
Interfaces y visibilidad de los assemblies 26
Justificación de la división 27
Independencia de librerías 28
Uso de IA Generativa y autoría 29
Documentación del uso de IA 29
Bitácora de decisiones de diseño 29
Anexo 30
Anexo A.1 30
Anexo A.2 31
Anexo B.1 32
Anexo B.2 36
Anexo B.3 39
Anexo B.4 41
Anexo B.5 44
Anexo B.6 46
Anexo B.7 47
Anexo B.8 51
Anexo B.9 51
Anexo C 52
Anexo D.1 53
Anexo D.2 54
Anexo D.3 55
Anexo D.4 55
Anexo D.5 56
Anexo E 56
Anexo F 57
F.1 — Frontera WebAPI–Servicios mediante DTOs (D-01) 57
F.2 — Fábrica para la composición de dependencias (D-34; actualiza D-06) 57
F.3 — Autenticación y autorización mediante un punto de extensión (D-13; D-14) 58
F.4 — Reglas de batalla en la entidad y sorteo inyectable (D-51) 58

F.5 — Cálculo de ELO reemplazable mediante Strategy (D-60) 59
F.6 — Rondas derivadas y emparejamiento por Strategy (D-62) 59
F.7 — Inscripción como agregado de reglas de elegibilidad (D-41; actualizado por D-67) 60
F.8 — Cambios multientidad con un solo guardado (D-42 y D-54) 60
F.9 — Coordinación de inscripciones, rondas y fallos concurrentes (D-70, D-71 y D-74) 61
F.10 — Formato uniforme y seguro de errores HTTP (D-07, D-10 y D-65) 61
F.11 — Contrato REST corregido según observaciones docentes (D-69 y D-72) 62
F.12 — Simplificación deliberada del código y pruebas sin depender de detalles internos (D-68) 62

Descripción general de la solución

FarmeORT: Batallas de Aura es una aplicación para gestionar torneos eliminatorios en los que los jugadores compiten por turnos utilizando skills de su repertorio. Contempla la gestión de usuarios, autenticación, catálogo de skills, torneos, inscripciones, rondas, batallas, lanzamientos, resultados, actualización del puntaje ELO y rankings.

Las reglas de negocio controlan las operaciones permitidas según cada rol y las transiciones válidas de estado, manteniéndose separadas de los mecanismos de exposición HTTP y persistencia.

FarmeORT y alcance de la primera entrega

La primera entrega comprende exclusivamente el backend de FarmeORT, expuesto mediante una API REST con 38 operaciones bajo /api/v1, sin interfaz gráfica.

El Jugador administra su repertorio, se inscribe o retira de torneos y participa en batallas. El Organizador gestiona los torneos asignados, sus skills obligatorias, rondas y fallos. El Administrador gestiona usuarios, skills y torneos, incluyendo la asignación de organizadores. Algunas consultas son públicas; las operaciones protegidas utilizan sesiones identificadas mediante tokens opacos enviados en el encabezado Authorization.

Los torneos evolucionan entre los estados Abierto, EnCurso, Finalizado y Cancelado. Las batallas contemplan cuatro lanzamientos por jugador, sin repetir skills y respetando las obligatorias. Su fallo determina el ganador, el avance en el torneo y la actualización del ELO.

La aplicación utiliza .NET 10 y ASP.NET Core, con SQL Server y Entity Framework Core Code First para la persistencia. Su organización en assemblies con responsabilidades diferenciadas se desarrolla en las siguientes secciones.

Estado de la solución, errores conocidos y supuestos

Las funcionalidades requeridas para el backend se encuentran implementadas. Permanecen tareas de preparación de la entrega, documentación y evidencias, sin que estas representen funcionalidades faltantes del backend.

Actualmente no se identifican errores funcionales conocidos aceptados para la entrega, situación que deberá verificarse nuevamente sobre el release final.

Existe una diferencia conocida entre la letra y la implementación: el Administrador puede generar rondas y finalizar torneos, aunque la consigna asigna dichas funciones al Organizador. Al no contar con confirmación docente de esta ampliación, se documenta como comportamiento implementado y no como requisito confirmado.

Arquitectura y organización en paquetes

FarmeORT se organiza en paquetes y assemblies con responsabilidades diferenciadas, separando las reglas del dominio, la coordinación de casos de uso, la exposición HTTP y la persistencia. Esta división busca favorecer la alta cohesión y el bajo acoplamiento, estableciendo dependencias explícitas mediante abstracciones y concentrando la selección de implementaciones concretas en la composición de la aplicación.

Organización según el modelo 4+1

El diseño se documenta siguiendo el modelo arquitectónico 4+1, que permite analizar la solución desde distintas perspectivas complementarias.

La vista lógica o de diseño presenta las clases, sus responsabilidades, relaciones y el modelo de datos. La vista de procesos describe mediante diagramas de interacción los mecanismos principales y la colaboración entre objetos durante la ejecución, incluyendo operaciones que requieren preservar la consistencia.

La vista de implementación representa los proyectos y assemblies, sus dependencias y contratos. La vista física contempla el backend Web API y SQL Server, mientras que los detalles de instalación y despliegue se documentan en los artefactos correspondientes.

Finalmente, la vista +1 de escenarios se representa mediante funcionalidades relevantes como la inscripción a un torneo, el lanzamiento de una skill y el fallo de una batalla. Estos escenarios permiten relacionar las distintas vistas arquitectónicas con el comportamiento real de la aplicación.

Diagrama general de paquetes

El Anexo A.1 — Diagrama general de paquetes presenta la descomposición lógica de FarmeORT y las dependencias entre sus nueve paquetes productivos: FarmeORT.Dominio, FarmeORT.DTOs, FarmeORT.Excepciones, FarmeORT.Fabrica, FarmeORT.IRepositorios, FarmeORT.IServicios, FarmeORT.Repositorios, FarmeORT.Servicios y FarmeORT.WebAPI.

El diagrama representa las dependencias lógicas derivadas del uso de tipos entre paquetes. Estas no deben confundirse necesariamente con las referencias físicas directas entre proyectos, que se documentan en la vista de implementación.

Complementariamente, el Anexo A.2 — Anidamiento de paquetes presenta los namespaces internos y su pertenencia a los paquetes principales, utilizando los nombres de la implementación y excluyendo los proyectos de pruebas.

Organización por capas y dependencias

Las dependencias se orientan de manera que las reglas centrales permanezcan independientes de los detalles de exposición HTTP y persistencia.

FarmeORT.WebAPI depende de FarmeORT.DTOs, FarmeORT.IServicios, FarmeORT.Excepciones y FarmeORT.Fabrica. Los controllers consumen contratos de servicios y no referencian directamente sus implementaciones ni los repositorios.

FarmeORT.Fabrica conoce FarmeORT.Servicios y FarmeORT.Repositorios para registrar las implementaciones concretas, centralizando la composición de dependencias fuera de WebAPI.

FarmeORT.Servicios depende de FarmeORT.IServicios, FarmeORT.Dominio, FarmeORT.IRepositorios y FarmeORT.Excepciones. Coordina los casos de uso y accede a la persistencia mediante interfaces, sin depender de Entity Framework Core.

FarmeORT.Repositorios depende de FarmeORT.IRepositorios y FarmeORT.Dominio, implementando el acceso a datos. A su vez, IServicios depende de DTOs, mientras que IRepositorios depende de Dominio.

Finalmente, FarmeORT.Dominio depende únicamente de FarmeORT.Excepciones, manteniéndose independiente de ASP.NET Core, Entity Framework Core y SQL Server. DTOs y Excepciones no poseen dependencias hacia otros proyectos de FarmeORT.

Esta dirección de dependencias evita trasladar detalles tecnológicos hacia las reglas de negocio y permite reemplazar implementaciones sin modificar necesariamente sus consumidores.

Responsabilidades de cada paquete

FarmeORT.Dominio. Contiene las entidades, estados y comportamientos propios del dominio de FarmeORT. Es responsable de las reglas que pueden resolverse a partir del estado de los propios objetos y también contiene las abstracciones correspondientes a comportamientos del dominio que poseen puntos de variación, como el cálculo del ELO y el emparejamiento.

FarmeORT.DTOs. Define los objetos utilizados para intercambiar información a través de los límites de la aplicación. Incluye las representaciones de entrada y salida utilizadas por la API y evita utilizar directamente las entidades del dominio como contrato HTTP.

FarmeORT.Excepciones. Reúne las excepciones específicas mediante las cuales el dominio y los casos de uso comunican condiciones conocidas de error. Estas excepciones permiten distinguir problemas funcionales esperados de errores inesperados de infraestructura.

FarmeORT.IServicios. Define las interfaces de los casos de uso disponibles para la capa de exposición. Actúa como contrato entre WebAPI y las implementaciones de los servicios, permitiendo que los controllers dependan de abstracciones.

FarmeORT.Servicios. Implementa los casos de uso de la aplicación. Coordina reglas que involucran múltiples entidades o fuentes de información, utiliza los contratos de persistencia y determina el orden en que deben realizarse las validaciones y efectos de cada operación.

FarmeORT.IRepositorios. Define los contratos necesarios para consultar y persistir información. Permite que los servicios expresen qué operaciones de almacenamiento necesitan sin depender de Entity Framework Core ni de una implementación concreta de base de datos.

FarmeORT.Repositorios. Implementa el acceso a datos mediante Entity Framework Core. Contiene el contexto de persistencia, las configuraciones de las entidades, los repositorios y las migraciones necesarias para representar el modelo sobre SQL Server.

FarmeORT.WebAPI. Expone los casos de uso a través de HTTP. Contiene los controllers y los mecanismos propios del borde web, como autenticación, autorización y tratamiento de errores HTTP. Su responsabilidad es traducir entre el contrato HTTP y los servicios de aplicación, evitando incorporar reglas de negocio que pertenezcan a capas inferiores.

FarmeORT.Fabrica. Constituye el punto de composición de la solución. Registra las implementaciones concretas de servicios, repositorios y demás colaboradores utilizados por la aplicación y las vincula con las abstracciones correspondientes mediante inyección de dependencias.

Esta distribución concentra responsabilidades relacionadas dentro de cada paquete y limita sus dependencias externas. Las decisiones que fundamentan esta organización y su impacto sobre la mantenibilidad, extensibilidad y testeabilidad se desarrollan posteriormente en Justificación del diseño.

Vista lógica — clases y relaciones

La vista lógica presenta las clases, interfaces y relaciones que materializan las responsabilidades de los paquetes descriptos anteriormente. Los diagramas se incluyen en el Anexo B — Diagramas de clases, organizados por paquete para facilitar su lectura y correspondencia con el código.

Se utiliza la nomenclatura real de la implementación y notación UML para representar las asociaciones, multiplicidades, composiciones, generalizaciones y realizaciones de interfaces que resultan relevantes para comprender la solución.

Diagramas de clases por paquete

El Anexo B.1 — FarmeORT.Dominio presenta las principales abstracciones del dominio. Entre ellas se encuentran Usuario, Skill, SkillDeclarada, Torneo, Inscripcion, Batalla, Lanzamiento y Sesion, junto con los estados y contratos de comportamiento asociados. Las relaciones permiten observar, entre otros aspectos, el repertorio de skills de un jugador, su participación mediante inscripciones, la pertenencia de las batallas a un torneo, los participantes de cada batalla y los lanzamientos que conforman su desarrollo.

Dentro del mismo paquete se representan además los puntos de variación propios del dominio mediante ICalculadorElo y IEstrategiaEmparejamiento, implementados actualmente por CalculadorEloBase y EmparejamientoPorElo. También se incluye ISorteo, utilizado por Batalla para resolver el jugador inicial cuando ambos participantes poseen el mismo ELO. Estas relaciones se representan como realizaciones de interfaces y no como jerarquías de herencia.

El Anexo B.2 — FarmeORT.DTOs presenta las clases utilizadas como contratos de entrada y salida de la API. Los DTOs se encuentran agrupados según el recurso al que pertenecen — usuarios, sesiones, skills, repertorios, torneos, inscripciones, rondas, batallas, lanzamientos, finalizaciones y ranking— y permiten mantener separado el contrato HTTP de las entidades internas del dominio.

El Anexo B.3 — FarmeORT.IServicios presenta las interfaces mediante las cuales la capa WebAPI accede a los casos de uso. Entre ellas se encuentran los contratos de usuarios, sesiones, skills, repertorios, torneos, skills obligatorias, inscripciones, rondas, batallas, lanzamientos, fallos, finalizaciones y ranking, además de abstracciones relacionadas con generación de tokens y hashing de contraseñas.

El Anexo B.4 — FarmeORT.Servicios muestra las implementaciones de estos contratos y sus principales colaboradores. Cada servicio se encuentra orientado a un conjunto acotado de casos de uso y depende de repositorios y elementos del dominio según la información y los comportamientos requeridos por cada operación.

El Anexo B.5 — FarmeORT.IRepositorios presenta las interfaces que definen las operaciones de persistencia requeridas por los servicios. Los contratos se organizan según los principales agregados o recursos persistentes: usuarios, sesiones, skills, torneos, inscripciones y batallas. También se incluyen los objetos utilizados para expresar filtros y resultados específicos de las consultas.

El Anexo B.6 — FarmeORT.Repositorios muestra las implementaciones concretas de los contratos de persistencia y su relación con FarmeORTContext. Este último constituye el contexto de Entity Framework Core utilizado para mapear y persistir las entidades del dominio.

El Anexo B.7 — FarmeORT.WebAPI presenta los controllers que exponen los recursos HTTP y los filtros utilizados para resolver responsabilidades transversales como autenticación, autorización y manejo de excepciones. Los controllers dependen de las interfaces de servicio correspondientes, manteniendo el borde HTTP separado de las implementaciones de los casos de uso.

El Anexo B.8 — FarmeORT.Excepciones presenta las excepciones específicas utilizadas para comunicar condiciones conocidas del dominio y de los casos de uso. Su separación permite representar explícitamente los distintos motivos de rechazo sin convertirlos en valores especiales o respuestas HTTP dentro de las capas inferiores.

Finalmente, el Anexo B.9 — FarmeORT.Fabrica presenta RegistroDependencias, responsable de vincular las abstracciones utilizadas por la aplicación con sus implementaciones concretas durante la composición del sistema.

Esta división permite analizar la estructura interna de cada paquete sin recurrir a un único diagrama excesivamente complejo. En particular, el diagrama de Dominio muestra las asociaciones centrales de la competencia y los puntos de variación, mientras que los restantes permiten comprender la separación entre contratos e implementaciones.

Jerarquías de herencia relevantes

FarmeORT utiliza herencia de forma acotada, priorizando interfaces y composición para desacoplar comportamientos.

En DTOs, UsuarioAdminRequest hereda de RegistroJugadorRequest, reutilizando los datos comunes del usuario y agregando el rol. Asimismo, TorneoDetalleResponse hereda de TorneoResponse para extender la representación básica del torneo con información detallada.

En WebAPI, AutenticacionOpcionalAttribute y AutorizacionAttribute especializan AutenticacionAttribute, reutilizando el mecanismo común de autenticación y adaptándolo a los distintos requisitos de autorización.

En Excepciones, ExcepcionConDetalles permite especializar errores que transportan información adicional, como SkillsObligatoriasFaltantesException y JugadoresSinSkillObligatoriaException.

También existen relaciones de herencia propias de los frameworks, como ControllerBase, DbContext y Migration, utilizadas como puntos de integración con ASP.NET Core y Entity Framework Core.

Estas relaciones se diferencian de las realizaciones de interfaces, como CalculadorEloBase respecto de ICalculadorElo y EmparejamientoPorElo respecto de IEstrategiaEmparejamiento.

Estas últimas permiten sustituir comportamientos a través de contratos, como se desarrolla posteriormente en la sección Patrones de diseño.

Modelo de datos y persistencia

FarmeORT utiliza SQL Server y Entity Framework Core con enfoque Code First. El modelo relacional conserva el estado de las entidades del dominio y refuerza sus invariantes mediante claves, índices y restricciones. El acceso a datos se encapsula en FarmeORT.Repositorios, evitando que el dominio y los servicios dependan directamente de la tecnología de persistencia.

Modelo de tablas de la base de datos

El Anexo C — Modelo de tablas de la base de datos presenta las tablas, sus claves primarias y foráneas, asociaciones y cardinalidades.

El modelo se compone de nueve tablas principales:

- Usuarios y Sesiones: almacenan los usuarios, roles, ELO, baja lógica y sesiones autenticadas. El email y nickname poseen índices únicos, mientras que Sesiones utiliza el token como clave primaria y referencia al usuario.
- Skills y SkillsDeclaradas: representan el catálogo global y el repertorio de cada jugador. Skills posee un código único y SkillsDeclaradas utiliza la clave compuesta JugadorId–SkillId, almacenando además el nivel y la fecha de declaración.
- Torneos y SkillsObligatorias: conservan el estado, organizador, campeón y código único del torneo. SkillsObligatorias representa la asociación muchos a muchos mediante la clave compuesta TorneoId–SkillId.
- Inscripciones: relaciona jugadores y torneos, conservando el estado y la fecha de inscripción.
- Batallas y Lanzamientos: registran los enfrentamientos, participantes, ronda, turnos, estado, fallo y variaciones de ELO, así como cada skill lanzada por jugador.

Las rondas no poseen una tabla independiente: se identifican mediante el número de ronda de las batallas. Los fallos se almacenan en Batallas, mientras que los rankings se calculan a partir del ELO y los resultados persistidos. Estas decisiones evitan duplicar información derivable del modelo existente.

Persistencia con Entity Framework Core

FarmeORTContext extiende DbContext y configura el modelo mediante Fluent API. El esquema cuenta con once migraciones que definen claves, relaciones, longitudes, conversiones de enumeraciones, índices, restricciones y comportamientos de eliminación.

Las entidades mantienen setters privados y operaciones de dominio, exponiendo sus colecciones como solo lectura para controlar las modificaciones. Los repositorios encapsulan las consultas y los guardados, utilizando AsNoTracking para lecturas cuando corresponde y seguimiento de entidades para operaciones que modifican su estado.

Integridad y consistencia

Las invariantes se protegen tanto mediante reglas del dominio como mediante restricciones de base de datos. Los índices únicos garantizan la unicidad del email, nickname y códigos de

skills y torneos, incluso frente a solicitudes concurrentes. Los conflictos de unicidad se traducen desde los repositorios a excepciones funcionales.

Las claves compuestas impiden duplicar skills declaradas u obligatorias. En Lanzamientos, índices únicos evitan repetir un número de turno o utilizar dos veces la misma skill por jugador y batalla; una restricción adicional limita los turnos al rango de 1 a 8.

Las relaciones utilizan eliminación en cascada cuando los registros dependientes carecen de sentido sin su propietario y Restrict cuando debe preservarse información histórica. Por esta razón, los usuarios poseen baja lógica, conservando sus relaciones con torneos, inscripciones y batallas.

Estas restricciones complementan las validaciones de dominio, protegiendo la integridad incluso cuando varias operaciones intentan modificar simultáneamente la información.

Consistencia, atomicidad y concurrencia en persistencia

Los repositorios utilizados durante una solicitud comparten un mismo FarmeORTContext con lifetime Scoped. Esto permite confirmar mediante un único SaveChanges los cambios relacionados, aprovechando la transacción de guardado de Entity Framework Core.

Por ejemplo, la cancelación de un torneo actualiza su estado y cancela las inscripciones y batallas correspondientes. El fallo de una batalla confirma conjuntamente el resultado, los ELO de ambos jugadores y la eliminación del perdedor. La generación de una ronda persiste todas sus batallas y, cuando corresponde, el cambio del torneo a EnCurso. Así se evita dejar operaciones parcialmente confirmadas.

Para la coordinación entre el inicio de sesión y la baja lógica de usuarios se utilizan transacciones explícitas con aislamiento Serializable, impidiendo confirmar una sesión concurrentemente con la eliminación lógica de su propietario.

Además, FarmeORT implementa concurrencia optimista mediante tres mecanismos:

- Torneo.Version: propiedad shadow de EF Core, de tipo Guid, configurada como token de concurrencia. Se actualiza ante cambios del torneo y operaciones que afectan a sus participantes, detectando modificaciones sobre estados desactualizados.
- Batalla.Estado: token que detecta operaciones concurrentes incompatibles sobre una misma batalla.
- Usuario.Elo: permite detectar modificaciones concurrentes del puntaje durante el fallo, evitando confirmar cálculos basados en valores anteriores.

Ante un conflicto, Entity Framework Core genera una excepción de concurrencia que los repositorios traducen a excepciones funcionales, como TorneoModificadoException o BatallaModificadaException.

No se realizan reintentos automáticos de operaciones de negocio: el conflicto se informa para que el cliente consulte nuevamente el estado vigente.

La combinación de atomicidad, restricciones relacionales y concurrencia optimista permite preservar la consistencia sin introducir bloqueos permanentes ni exponer detalles de infraestructura a las capas superiores.

Vista de procesos — interacciones relevantes

La vista de procesos complementa la estructura estática presentada anteriormente, mostrando cómo colaboran los objetos durante la ejecución de operaciones relevantes de FarmeORT. Los diagramas de interacción correspondientes se presentan en el Anexo D — Diagramas de interacción.

Se seleccionaron tres casos de uso de negocio que atraviesan distintas capas de la aplicación y permiten comprender la distribución de responsabilidades entre controllers, servicios, entidades del dominio y repositorios. Adicionalmente, se incluyen el inicio de sesión y el manejo de excepciones, que permiten observar mecanismos relevantes de autenticación, persistencia y traducción de errores al contrato HTTP.

En los tres casos de uso de negocio, la operación comienza con los controles de autenticación, autorización y validación correspondientes. Estos pasos se representan para mostrar el recorrido completo de la solicitud, mientras que las interacciones posteriores se concentran en las reglas y decisiones específicas de cada funcionalidad. El inicio de sesión, en cambio, permite observar cómo se generan y persisten las credenciales utilizadas posteriormente en las operaciones protegidas.

El objetivo no es representar todas las operaciones de la API, sino seleccionar aquellas interacciones que permiten comprender los mecanismos más relevantes del diseño y su correspondencia con la implementación.

Criterio de selección

Se seleccionaron tres casos de uso de negocio: inscripción a un torneo, lanzamiento de una skill y fallo de una batalla. Adicionalmente, se incorporaron el inicio de sesión y el manejo de excepciones, por representar mecanismos relevantes para comprender la autenticación, la consistencia y el tratamiento transversal de errores.

La inscripción a un torneo permite observar cómo un servicio coordina información proveniente del torneo, del jugador y de las inscripciones existentes antes de crear una nueva entidad. También permite distinguir las validaciones que requieren información de varios objetos de aquellas que pertenecen a las invariantes propias de Inscripcion.

El lanzamiento de una skill representa la operación central de una batalla. Su interés reside en que ServicioLanzamientos coordina la obtención de los objetos necesarios, mientras que Batalla concentra las principales reglas de participación, turnos, repertorio y skills obligatorias.

El fallo de una batalla permite representar una operación con múltiples efectos relacionados: validación del resultado, cálculo y actualización del ELO de ambos jugadores, registro del ganador y eliminación del perdedor. Estos cambios deben persistirse de forma conjunta, manteniendo la consistencia frente a posibles conflictos de concurrencia.

El inicio de sesión permite observar la separación de responsabilidades entre la validación de credenciales, la generación del token y la persistencia de la sesión. También representa la coordinación transaccional necesaria para evitar confirmar sesiones de usuarios eliminados

durante operaciones concurrentes. A diferencia de los casos anteriores, se trata de una operación pública que establece las credenciales utilizadas en solicitudes posteriores.

Finalmente, el manejo de excepciones muestra una responsabilidad transversal del borde HTTP: transformar condiciones funcionales conocidas en respuestas uniformes, sin trasladar códigos de estado HTTP al dominio o a los servicios.

Estos diagramas fueron seleccionados por su aporte a la comprensión de responsabilidades, colaboración entre objetos, encapsulamiento de reglas y mecanismos de consistencia, evitando documentar operaciones cuya interacción no agrega información significativa sobre el diseño.

Inscripción a un torneo

El Anexo D.1 — Inscripción a un torneo representa la interacción iniciada por InscripcionesController al recibir una solicitud de inscripción de un Jugador autenticado.

Una vez obtenida la identidad del jugador desde la sesión, el controller delega el caso de uso en IServicioInscripciones. La implementación ServicioInscripciones obtiene en primer lugar el torneo mediante IRepositorioTorneos y posteriormente al jugador, junto con su repertorio, mediante IRepositorioUsuarios.

El servicio valida que el torneo se encuentre Abierto, que el jugador no posea ya una inscripción Aceptada en ese torneo y que todavía exista lugar dentro del cupo. Estas validaciones requieren información que excede el estado de una única entidad y, por lo tanto, son coordinadas desde el servicio.

Una vez satisfechas esas precondiciones se crea una instancia de Inscripcion. Su constructor verifica las reglas que puede resolver utilizando el torneo y el jugador recibidos, en particular que el torneo continúe abierto y que el jugador tenga declaradas todas las skills obligatorias requeridas.

La nueva inscripción se entrega a IRepositorioInscripciones, que la agrega al contexto y confirma el cambio. El repositorio también participa en la coordinación de concurrencia del torneo, de modo que una inscripción preparada sobre un estado que quedó desactualizado no pueda confirmarse silenciosamente.

Finalmente, ServicioInscripciones construye el InscripcionResponse y lo devuelve al controller, que responde 201 Created e identifica la inscripción creada mediante el encabezado Location.

Este flujo permite distinguir las responsabilidades de coordinación del servicio de las invariantes propias de Inscripcion y del mecanismo de persistencia utilizado por el repositorio.

Lanzamiento de una skill

El Anexo D.2 — Lanzamiento de una skill representa la operación principal que ejecuta un Jugador durante una batalla.

LanzamientosController obtiene el identificador del jugador desde la sesión autenticada y delega la operación en ServicioLanzamientos. El servicio obtiene la batalla mediante IRepositorioBatallas, al jugador junto con su repertorio mediante IRepositorioUsuarios y la skill solicitada mediante IRepositorioSkills.

Con estas referencias, el servicio invoca Batalla.Lanzar. La entidad Batalla concentra las reglas correspondientes al desarrollo de un turno. Verifica que la batalla se encuentre Pendiente, que el jugador participe en ella y que sea efectivamente su turno. Luego comprueba que la skill forme parte del repertorio del jugador, que no haya sido utilizada previamente por él en la misma batalla y que el lanzamiento no impida cumplir con las skills obligatorias pendientes del torneo.

Si las condiciones son válidas, Batalla crea una instancia de Lanzamiento, la incorpora a su colección y actualiza el jugador al que corresponde el siguiente turno. Cuando se registra el octavo lanzamiento, cambia su estado a EnArbitraje y deja de existir un jugador con turno pendiente.

Una vez modificada la entidad, ServicioLanzamientos solicita a IRepositorioBatallas que confirme el cambio. Las restricciones de persistencia y el token de concurrencia del estado de la batalla complementan las validaciones realizadas por el dominio frente a solicitudes simultáneas incompatibles.

Finalmente, el servicio devuelve un LanzamientoResultadoResponse con el número de turno, el estado actualizado de la batalla y, cuando corresponde, el identificador del jugador que debe realizar el siguiente lanzamiento. El controller responde 201 Created.

Este flujo muestra cómo una operación coordinada desde un servicio puede delegar la mayor parte de sus reglas a la entidad que posee la información necesaria para evaluarlas.

Fallo de batalla y actualización de ELO

El Anexo D.3 — Fallo de batalla y actualización de ELO representa uno de los casos de uso con mayor cantidad de efectos coordinados dentro de FarmeORT.

FallosController obtiene el identificador del Organizador autenticado y delega la operación en ServicioFallos. El servicio consulta la batalla mediante IRepositorioBatallas y verifica que pertenezca a un torneo asignado al Organizador que realiza la solicitud.

A continuación, Batalla.ValidarFallo comprueba que el estado permita fallarla, que el jugador indicado como ganador sea uno de sus participantes y que el comentario cumpla las condiciones requeridas.

Luego el servicio determina las referencias al ganador y al perdedor y consulta mediante IRepositorioInscripciones la inscripción Aceptada del jugador derrotado. Esta validación se realiza antes de modificar el ELO, evitando producir cambios parciales cuando el estado de la competencia no permite completar el fallo.

El cálculo de los nuevos puntajes se delega a ICalculadorElo. La implementación configurada, CalculadorEloBase, recibe los ELO previos del ganador y del perdedor y devuelve ambos valores nuevos. ServicioFallos actualiza los dos jugadores y calcula las variaciones que deben quedar registradas en la batalla.

Posteriormente se invoca Batalla.Fallar, que registra ganador, comentario, variaciones de ELO y cambia el estado a Finalizada. La inscripción del perdedor recibe Eliminar, pasando a estado Eliminada.

Aunque intervienen varias entidades y dos contratos de repositorio, todos los objetos participan del mismo FarmeORTContext scoped. ServicioFallos realiza finalmente un único guardado a través de IRepositorioBatallas, confirmando en una misma unidad el resultado de la batalla, los dos ELO, la eliminación del perdedor y los tokens de concurrencia relacionados.

Si durante el proceso otra solicitud modifica de manera incompatible la batalla, el ELO o el torneo, el conflicto es detectado al persistir y la operación no deja efectos parciales.

Una vez confirmado el fallo, el servicio construye el detalle actualizado de la batalla y el controller responde 201 Created, identificando mediante Location la batalla sobre la que se registró el fallo.

Manejo de excepciones

El Anexo D.4 - Manejo de excepciones representa el mecanismo mediante el cual FarmeORT transforma las excepciones producidas durante la ejecución de los casos de uso en respuestas HTTP uniformes.

Cuando una excepción es capturada por FiltroExcepciones, el filtro consulta el mapeo correspondiente a su tipo para determinar el código de estado HTTP, el código de error y el mensaje que debe devolverse al cliente. Si el mapeo no establece un mensaje específico, se utiliza el mensaje de la excepción. Cuando se trata de una ExcepcionConDetalles, también se recupera la información adicional asociada al error.

A partir de estos datos se construye un ErrorResponse, que contiene el mensaje, el código de error y los detalles cuando corresponde. El filtro utiliza este objeto para generar un ObjectResult con el estado HTTP determinado. Si la excepción no posee un mapeo conocido, se devuelve un error 500 INTERNAL_ERROR con un mensaje genérico, evitando exponer información técnica de la aplicación.

Este mecanismo se complementa con RespuestaValidacion, responsable de las respuestas 400 por solicitudes inválidas, y con los filtros de autenticación y autorización, que generan directamente los errores 401 y 403 previstos. Las excepciones inesperadas fuera del alcance del filtro MVC se atienden mediante UseExceptionHandler, mientras que UseStatusCodePages permite completar respuestas de error sin cuerpo.

La separación permite que las entidades del dominio y los servicios comuniquen condiciones funcionales mediante excepciones propias, sin depender de HTTP. La responsabilidad de traducirlas a un contrato de respuesta uniforme permanece concentrada en WebAPI, favoreciendo la cohesión y evitando duplicar el tratamiento de errores en cada controller.

Inicio de Sesión

El Anexo D.5 — Inicio de sesión representa la interacción mediante la cual FarmeORT autentica a un usuario y genera las credenciales necesarias para acceder a las operaciones protegidas de la API.

SesionesController recibe una solicitud POST /api/v1/sesiones con el email y la contraseña. Al tratarse de una operación pública, no requiere autenticación previa. ASP.NET Core valida el contenido recibido mediante LoginRequest y, si es válido, el controller delega la operación en IServicioSesiones.

La implementación ServicioSesiones obtiene el usuario mediante IRepositorioUsuarios y verifica que exista y no esté eliminado. Posteriormente utiliza IHasheadorContrasenas para comprobar la contraseña ingresada contra el hash almacenado. Si las credenciales son incorrectas o el usuario fue eliminado, se lanza CredencialesInvalidasException, que se traduce en una respuesta 401 INVALID_CREDENTIALS sin revelar cuál de las verificaciones falló.

Una vez autenticado el usuario, el servicio solicita a IGeneradorTokens un token de tipo Guid y crea una instancia de Sesion, asociándola con el usuario. La persistencia se delega a IRepositorioSesiones, que inicia una transacción con aislamiento Serializable y realiza una escritura condicionada sobre el usuario para coordinar el inicio de sesión con su posible baja lógica concurrente.

Si el usuario ya no se encuentra activo, se rechaza la operación sin persistir la sesión. En caso contrario, el repositorio incorpora la sesión al contexto, ejecuta SaveChanges y confirma la transacción.

Finalmente, ServicioSesiones construye un SesionResponse con el token, el identificador del usuario y su rol. SesionesController responde 201 Created e identifica la sesión creada mediante el encabezado Location.

Este flujo permite distinguir la responsabilidad de autenticación del servicio, la generación de credenciales mediante abstracciones y las garantías de consistencia aportadas por la persistencia. La coordinación transaccional evita confirmar una nueva sesión cuando la baja lógica concurrente del usuario ya fue efectuada.

Justificación del diseño

Las decisiones de diseño de FarmeORT buscan distribuir las responsabilidades de forma que los distintos motivos de cambio queden localizados y que los componentes centrales no dependan de detalles tecnológicos que puedan variar. Los principios y patrones utilizados no se aplicaron como objetivos en sí mismos, sino para resolver problemas concretos de acoplamiento, extensibilidad, testeabilidad y mantenimiento.

Principios SOLID

El principio de Responsabilidad Única (SRP) se refleja tanto en la división entre assemblies como dentro de ellos. Los controllers se limitan a actuar como borde HTTP, obtener la información de la request y delegar en los servicios. Los servicios coordinan casos de uso, las entidades del dominio protegen las reglas que pueden resolver con su propio estado y los repositorios concentran el acceso a datos.

Esta separación también se aplica dentro de los casos de uso. Por ejemplo, la generación de rondas, el lanzamiento de skills, el fallo de batallas y la finalización de torneos se encuentran implementados en servicios diferentes. Aunque todos pertenecen al mismo dominio, poseen motivos de cambio distintos. Mantenerlos separados evita que modificar una regla de lanzamiento obligue a intervenir sobre el código encargado de calcular ELO o finalizar un torneo.

El principio de Abierto/Cerrado (OCP) resulta especialmente relevante en los puntos donde la consigna anticipa posibles variaciones. ServicioFallos no contiene directamente la fórmula de cálculo del ELO, sino que depende de ICalculadorElo. La implementación actual es CalculadorEloBase, pero una nueva política puede incorporarse mediante otra implementación sin modificar el caso de uso que falla una batalla.

El mismo criterio se utiliza en la generación de rondas. ServicioRondas depende de IEstrategiaEmparejamiento, mientras que EmparejamientoPorElo implementa el criterio utilizado actualmente. Una forma alternativa de emparejar jugadores puede incorporarse sustituyendo la estrategia registrada, sin alterar la coordinación de la ronda.

El principio de Sustitución de Liskov (LSP) se considera en los puntos de variación definidos mediante contratos. ServicioFallos utiliza ICalculadorElo y ServicioRondas utiliza IEstrategiaEmparejamiento, sin depender de detalles exclusivos de sus implementaciones actuales. Esto permite incorporar estrategias alternativas siempre que respeten las operaciones, precondiciones y resultados esperados por sus consumidores. La sustitución no depende únicamente de implementar una interfaz, sino también de preservar el comportamiento comprometido por el contrato.

La Segregación de Interfaces (ISP) se observa en la utilización de contratos específicos en lugar de interfaces generales que agrupen responsabilidades no relacionadas. Existen, por ejemplo, IServicioLanzamientos, IServicioFallos, IServicioRondas e IServicioRanking, además de interfaces de repositorio separadas por los recursos persistentes principales. Esto permite que cada cliente dependa únicamente de las operaciones que necesita.

Finalmente, el principio de Inversión de Dependencias (DIP) determina gran parte de la dirección de las dependencias de la solución. WebAPI utiliza interfaces de IServicios en lugar

de depender de las implementaciones de Servicios, y los servicios utilizan IRepositorios en lugar de depender de Entity Framework Core. De esta forma, los módulos encargados de las reglas y casos de uso no quedan atados a los detalles de persistencia.

GRASP

La distribución de responsabilidades también puede analizarse utilizando los principios GRASP.

Information Expert se aplica asignando una responsabilidad al objeto que posee la información necesaria para resolverla. Batalla decide si un jugador puede realizar un lanzamiento porque conoce sus participantes, el turno actual, los lanzamientos previos y las skills obligatorias del torneo. También valida si puede ser fallada. Torneo, por su parte, controla las transiciones que dependen de su propio estado, como el inicio de la competencia.

El principio Creator se observa en la creación de Lanzamiento por parte de Batalla. Una batalla contiene y registra sus lanzamientos y posee la información necesaria para determinar el número de turno y asociarlo con el jugador y la skill correspondiente, por lo que resulta el objeto adecuado para crearlos.

En términos de Controller, los servicios de aplicación reciben y coordinan las operaciones del sistema. Este concepto no debe confundirse con los controllers de ASP.NET Core: estos últimos forman parte del borde HTTP, mientras que clases como ServicioInscripciones, ServicioRondas o ServicioFallos coordinan el caso de uso propiamente dicho.

Low Coupling y High Cohesion fundamentan la separación entre dominio, servicios, contratos, persistencia y WebAPI. Cada componente concentra responsabilidades relacionadas y conoce solamente los elementos necesarios para cumplirlas. En particular, un cambio en SQL Server o Entity Framework Core no debería requerir modificar las reglas del dominio.

Polymorphism y Protected Variations se utilizan en los puntos donde existe una variación prevista. Las políticas de ELO y emparejamiento se encuentran protegidas detrás de interfaces, evitando que una modificación en el algoritmo se propague hacia los servicios que las consumen.

También se aplica Indirection mediante las interfaces de servicios y repositorios. Estas abstracciones introducen un intermediario entre componentes que de otra forma quedarían directamente acoplados. IRepositorioBatallas, por ejemplo, permite que ServicioFallos exprese qué necesita persistir sin conocer cómo se implementa esa persistencia.

Finalmente, servicios y repositorios pueden entenderse como casos de Pure Fabrication. No representan conceptos propios del dominio de las batallas de Aura, pero se introducen para concentrar responsabilidades técnicas y de coordinación que no pertenecen naturalmente a una entidad, favoreciendo cohesión y bajo acoplamiento.

Inyección de dependencias

La aplicación utiliza el contenedor de inyección de dependencias provisto por .NET para proporcionar a cada componente sus colaboradores.

Los controllers reciben interfaces de servicios; los servicios reciben interfaces de repositorios y las estrategias que necesitan; y los repositorios reciben el FarmeORTContext. De esta forma, las clases no construyen por sí mismas sus dependencias concretas mediante new, sino que declaran los contratos que necesitan.

Los servicios, repositorios, FarmeORTContext, estrategias de ELO y emparejamiento y el mecanismo de sorteo se registran con lifetime Scoped, por lo que durante una request comparten el mismo alcance. Esto es especialmente importante para la persistencia: los repositorios utilizados dentro de un mismo caso de uso reciben el mismo contexto y pueden participar en una misma unidad de guardado.

TimeProvider.System se registra como Singleton. Los servicios que necesitan conocer la fecha u hora dependen de TimeProvider, lo que evita utilizar directamente el reloj del sistema dentro de la lógica y permite sustituirlo durante las pruebas.

La principal ventaja de esta decisión es reducir el acoplamiento hacia implementaciones concretas y mejorar la testeabilidad. Su costo es incorporar una configuración explícita de las dependencias y requerir que el equipo pueda identificar correctamente el lifetime apropiado para cada objeto.

Fábrica y composición

El assembly FarmeORT.Fabrica actúa como punto de composición de la aplicación. Su clase pública RegistroDependencias centraliza la conexión entre la WebAPI y los assemblies que contienen las implementaciones de servicios y repositorios.

RegistroDependencias.AgregarDependencias delega el registro concreto en RegistroServicios y RegistroRepositorios. De esta forma, WebAPI necesita conocer FarmeORT.Fabrica, pero no mantener referencias directas hacia FarmeORT.Servicios y FarmeORT.Repositorios.

Aunque el proyecto se denomina Fabrica, su responsabilidad principal en la solución actual es la de composition root: determinar qué implementaciones concretas se utilizan para satisfacer las abstracciones. No se presenta como una aplicación de Factory Method, ya que no existe una jerarquía donde una subclase redefina qué producto crear. La creación de los objetos es realizada por el contenedor de inyección de dependencias.

Esta decisión evita incorporar una fábrica artificial únicamente para utilizar un patrón y mantiene centralizada la configuración de las dependencias físicas de la aplicación.

Patrones de diseño

Strategy para el cálculo de ELO

El cálculo de ELO constituye un punto de variación requerido explícitamente por la consigna: debe ser posible incorporar otra regla sin modificar el código encargado de finalizar una batalla.

Se consideró implementar la fórmula directamente dentro de ServicioFallos. Esta alternativa era más simple inicialmente, pero acoplaba el caso de uso con una política que se sabe que

puede variar. También se consideró Template Method, pero la variación podría involucrar una fórmula completamente distinta y no únicamente un paso de un algoritmo común.

Se optó por Strategy mediante ICalculadorElo y CalculadorEloBase. ServicioFallos conoce únicamente la interfaz y recibe la estrategia mediante inyección de dependencias.

La ventaja principal es que una nueva política de ELO se incorpora como una nueva implementación y un cambio en la composición, sin modificar ServicioFallos. Esto favorece OCP, DIP, Polymorphism y Protected Variations. Como costo, el diseño incorpora una abstracción adicional, que se justifica porque la variación fue identificada explícitamente como requisito del sistema.

Strategy para el emparejamiento

La generación de rondas presenta otro punto de variación: el criterio de emparejamiento debe ser determinístico y estar documentado, pero puede cambiar.

Una alternativa era implementar el ordenamiento y creación de parejas como un método privado de ServicioRondas. Esa solución era más directa, pero dejaba al servicio cerrado sobre una única política.

Se utilizó nuevamente Strategy mediante IEstrategiaEmparejamiento. La implementación actual, EmparejamientoPorElo, ordena por ELO descendente, desempata por identificador y enfrenta al mejor posicionado con el peor, luego al segundo con el penúltimo y así sucesivamente.

El servicio se limita a solicitar los pares. De esta manera, cambiar a otro criterio no modifica la coordinación del caso de uso.

Como costo, esta solución incorpora una interfaz, una implementación y configuración adicional en el contenedor de dependencias. Se aceptó esa complejidad porque el criterio de emparejamiento constituye un punto de variación real y su aislamiento evita modificar el caso de uso cuando cambie la política.

Template Method en autenticación y autorización

La seguridad HTTP utiliza una aplicación acotada de Template Method.

AutenticacionAttribute define el procedimiento general: obtener el Bearer, validar el token, obtener la sesión autenticada y almacenarla en el HttpContext. Al finalizar ejecuta el punto de extensión AutorizarSesion.

AutorizacionAttribute hereda de esta clase y redefine únicamente ese paso para verificar que el rol del usuario pertenezca al conjunto permitido. AutenticacionOpcionalAttribute especializa el comportamiento permitiendo que determinadas operaciones continúen sin sesión cuando no se envía un token.

La alternativa de utilizar filtros completamente independientes hubiera repetido la autenticación o requerido coordinar explícitamente su orden. La jerarquía utilizada garantiza que no pueda ejecutarse una autorización por rol sin haber resuelto previamente la autenticación.

El costo de esta decisión es introducir acoplamiento por herencia. Por este motivo se utiliza únicamente en una jerarquía pequeña donde existe un algoritmo común estable y puntos de extensión claramente definidos.

Acceso a datos

El acceso a datos se implementa mediante interfaces de repositorio ubicadas en FarmeORT.IRepositorios e implementaciones basadas en Entity Framework Core dentro de FarmeORT.Repositorios.

Los servicios expresan las operaciones que necesitan mediante interfaces como IRepositorioUsuarios, IRepositorioTorneos, IRepositorioInscripciones e IRepositorioBatallas. No conocen DbContext, consultas LINQ específicas de Entity Framework Core ni detalles de SQL Server.

Esta separación evita que las decisiones de persistencia se filtren hacia los casos de uso y permite mantener en los repositorios responsabilidades como consultas, tracking, traducción de conflictos de unicidad o concurrencia y ejecución de SaveChanges.

No se creó una abstracción genérica del tipo IRepositorio<T>, ya que los distintos recursos requieren consultas y comportamientos específicos. Utilizar contratos propios permite que cada interfaz exponga únicamente las operaciones necesarias para sus clientes, favoreciendo ISP y haciendo explícita la intención de cada consulta.

Manejo de excepciones

Las condiciones funcionales de error se representan mediante excepciones específicas del dominio y de los casos de uso. Esta decisión permite comunicar incumplimientos de reglas sin acoplar las capas inferiores a HTTP ni obligarlas a construir respuestas propias de la API.

Se decidió concentrar la traducción de estas excepciones en FiltroExcepciones, utilizando un mapeo explícito entre tipos de excepción, códigos HTTP, códigos de error y mensajes. Como alternativa, podría haberse realizado el tratamiento mediante bloques try/catch en cada controller, pero esto habría duplicado lógica y aumentado el acoplamiento entre los casos de uso y el contrato HTTP.

La solución también separa los errores según su origen: RespuestaValidacion atiende las solicitudes inválidas, los filtros de seguridad producen sus rechazos previstos y el middleware cubre errores inesperados fuera del alcance de MVC. Todos utilizan el mismo formato ErrorResponse, manteniendo la consistencia de las respuestas y evitando exponer información interna.

Esta organización favorece SRP y la mantenibilidad, ya que una modificación en la representación de los errores puede concentrarse en WebAPI sin alterar las reglas del dominio. Como contrapartida, el mecanismo requiere mantener actualizado el mapeo de excepciones cuando se incorporan nuevas condiciones funcionales.

Impacto sobre la mantenibilidad

En conjunto, estas decisiones buscan que los cambios tengan un alcance acotado.

Una nueva política de ELO o emparejamiento puede agregarse detrás de las interfaces existentes; un cambio en la tecnología de persistencia queda principalmente contenido en Repositorios; y una modificación en el formato HTTP se concentra en WebAPI y DTOs sin obligar a alterar las entidades del dominio.

La separación introduce más clases, interfaces y assemblies que una implementación monolítica. Ese costo se aceptó porque las fronteras utilizadas corresponden a responsabilidades y puntos de variación reales del obligatorio, y no a abstracciones creadas únicamente para aumentar la cantidad de capas o patrones.

Decisiones propias, mecanismos y algoritmos relevantes

Las rondas no poseen entidad persistente propia: se derivan del número de ronda de las batallas, evitando duplicar estado.

El emparejamiento es determinístico: ELO descendente, identificador como desempate y mejor contra peor. El cálculo de ELO utiliza los valores previos de ambos participantes antes de actualizarlos, con K = 32, redondeo entero y piso 100.

Las operaciones con múltiples efectos, como generar una ronda o fallar una batalla, se persisten atómicamente. Para modificaciones competidoras se utiliza concurrencia optimista sobre torneo, batalla y ELO; ante conflictos no se reintenta automáticamente, sino que se exige trabajar nuevamente sobre el estado vigente.

Vista de implementación

La vista de implementación representa cómo las responsabilidades lógicas descriptas anteriormente se materializan en proyectos y assemblies independientes. La solución productiva está compuesta por nueve assemblies, cada uno con un límite físico y un conjunto de dependencias explícitas.

El Anexo E — Diagrama de componentes representa estos assemblies, sus dependencias y las interfaces mediante las cuales colaboran. Los proyectos correspondientes a pruebas no forman parte de esta vista.

Estructura de assemblies

FarmeORT.Dominio contiene las entidades, enumeraciones y comportamientos propios del problema, así como los puntos de variación que pertenecen al dominio, entre ellos ICalculadorElo, IEstrategiaEmparejamiento e ISorteo. Su única dependencia de proyecto es FarmeORT.Excepciones.

FarmeORT.DTOs contiene los contratos utilizados para representar datos de entrada y salida. No depende de otros assemblies de FarmeORT.

FarmeORT.Excepciones contiene las excepciones funcionales compartidas por las capas que necesitan comunicar errores conocidos y no depende de otros proyectos de la solución.

FarmeORT.IServicios define los contratos correspondientes a los casos de uso que consume la capa WebAPI. Depende de FarmeORT.DTOs, ya que los métodos de estos contratos reciben y devuelven representaciones utilizadas en el límite de la aplicación.

FarmeORT.IRepositorios define los contratos de persistencia requeridos por los servicios. Depende de FarmeORT.Dominio, porque sus operaciones trabajan con las entidades y conceptos persistentes del dominio.

FarmeORT.Servicios implementa los contratos de FarmeORT.IServicios. Para coordinar los casos de uso depende además de FarmeORT.Dominio, FarmeORT.IRepositorios y FarmeORT.Excepciones.

FarmeORT.Repositorios implementa los contratos de FarmeORT.IRepositorios utilizando Entity Framework Core. Depende de FarmeORT.Dominio para mapear y persistir las entidades.

FarmeORT.Fabrica constituye el punto de composición entre abstracciones e implementaciones. Depende de FarmeORT.Servicios y FarmeORT.Repositorios y expone el mecanismo utilizado para registrar sus implementaciones en el contenedor de dependencias.

Finalmente, FarmeORT.WebAPI constituye el ejecutable y borde HTTP de la aplicación. Depende de FarmeORT.DTOs, FarmeORT.IServicios, FarmeORT.Excepciones y FarmeORT.Fabrica. No referencia directamente los assemblies FarmeORT.Servicios ni FarmeORT.Repositorios.

Diagrama de componentes

El Anexo E — Diagrama de componentes muestra la correspondencia entre los componentes físicos de la solución y los proyectos productivos que los implementan.

Las dependencias representadas en el diagrama coinciden con las referencias efectivas entre proyectos. De esta forma, la vista permite comprobar que las separaciones planteadas en el diseño no existen únicamente a nivel conceptual, sino que se encuentran reforzadas mediante límites físicos entre assemblies.

Las interfaces ubicadas en FarmeORT.IServicios y FarmeORT.IRepositorios constituyen puntos de comunicación especialmente relevantes entre los componentes. Las implementaciones concretas permanecen en FarmeORT.Servicios y FarmeORT.Repositorios, mientras que FarmeORT.Fabrica es responsable de vincularlas durante la composición de la aplicación.

Dependencias entre componentes

La dirección de las dependencias evita que los componentes centrales necesiten conocer los detalles de los componentes externos.

WebAPI conoce los contratos de servicios y las representaciones utilizadas en el protocolo HTTP, pero no necesita conocer las clases concretas que implementan los casos de uso ni la tecnología utilizada para persistir información.

De forma equivalente, Servicios depende de interfaces de repositorio y no de sus implementaciones. Esto permite que la coordinación de los casos de uso conozca qué operaciones de persistencia necesita sin conocer cómo se ejecutan.

Repositorios ocupa el límite donde las abstracciones de persistencia se implementan mediante Entity Framework Core y SQL Server. Como consecuencia, el conocimiento de estas tecnologías queda concentrado principalmente en este componente.

La dependencia de FarmeORT.Fabrica hacia Servicios y Repositorios es intencional: algún componente debe conocer las implementaciones concretas para poder construir el grafo de objetos. Concentrar este conocimiento en el punto de composición evita distribuirlo por WebAPI.

Interfaces y visibilidad de los assemblies

La visibilidad de los tipos fue utilizada como parte de los límites de cada assembly. No todas las clases se exponen públicamente: se mantienen internal aquellos elementos que solamente constituyen detalles de implementación del propio componente.

FarmeORT.IServicios y FarmeORT.IRepositorios exponen públicamente sus interfaces, ya que constituyen los contratos que deben ser consumidos desde otros assemblies.

En FarmeORT.Servicios, las implementaciones concretas como ServicioRondas, ServicioFallos o ServicioLanzamientos son internal. Desde otros componentes se consumen a

través de las interfaces correspondientes. La excepción relevante es RegistroServicios, que debe ser público para permitir que el punto de composición registre esas implementaciones.

Un criterio equivalente se aplica en FarmeORT.Repositorios: las implementaciones concretas de los repositorios son internal, mientras que RegistroRepositorios debe ser público para participar en la composición. FarmeORTContext también es público porque forma parte de la configuración necesaria para Entity Framework Core y del registro del contexto.

En FarmeORT.Fabrica, RegistroDependencias constituye la interfaz pública del componente. WebAPI utiliza este punto de entrada para configurar la aplicación sin depender directamente de Servicios ni Repositorios.

En FarmeORT.WebAPI, los controllers son públicos porque deben ser descubiertos por ASP.NET Core, mientras que filtros, extensiones y elementos auxiliares que no forman parte del contrato entre assemblies se mantienen internos.

FarmeORT.Dominio y FarmeORT.DTOs exponen aquellos tipos que necesariamente deben atravesar los límites de assembly. En Dominio esto incluye las entidades y abstracciones utilizadas por Servicios y Repositorios; en DTOs, las representaciones utilizadas por WebAPI e IServicios.

Esta utilización de la visibilidad permite reforzar físicamente la intención del diseño: un consumidor puede utilizar la interfaz ofrecida por un componente sin depender de detalles que deberían permanecer encapsulados dentro de él.

Justificación de la división

La división en assemblies busca que las fronteras conceptuales de la solución también tengan una representación física.

Mantener Dominio separado evita introducir dependencias hacia ASP.NET Core o Entity Framework Core en las reglas centrales. Separar los contratos de servicios de sus implementaciones permite que WebAPI dependa de abstracciones. Del mismo modo, separar IRepositorios de Repositorios evita que los servicios necesiten referenciar la implementación de persistencia.

DTOs se mantiene independiente del dominio para evitar convertir las entidades persistentes en contratos HTTP. Excepciones se separa porque sus tipos deben poder ser utilizados por distintas capas sin introducir dependencias en sentido contrario.

Finalmente, Fabrica concentra deliberadamente las dependencias hacia las implementaciones concretas. Sin este componente, WebAPI debería referenciar directamente Servicios y Repositorios únicamente para poder registrarlos, debilitando la separación física establecida por los contratos.

El costo de esta organización es una mayor cantidad de proyectos y referencias que en una solución monolítica. Se consideró aceptable porque las fronteras introducidas corresponden a responsabilidades y dependencias reales y permiten que el compilador contribuya a impedir dependencias que el diseño pretende evitar.

Independencia de librerías

La estructura busca limitar el impacto de las principales librerías y frameworks utilizados por la solución.

ASP.NET Core se concentra en FarmeORT.WebAPI, donde se encuentran los controllers, filtros y configuración propia del protocolo HTTP. Las entidades y reglas del dominio no requieren conocer conceptos como requests, responses, status codes o ControllerBase.

Entity Framework Core y el acceso concreto a SQL Server se concentran en FarmeORT.Repositorios. Los servicios se comunican con esta capa mediante interfaces propias y las entidades del dominio no necesitan conocer DbContext, consultas específicas de EF Core ni mecanismos HTTP.

La inyección de dependencias permite seleccionar las implementaciones concretas desde la composición de la aplicación. Por este motivo, cambiar un detalle de infraestructura no implica necesariamente modificar los componentes que expresan las reglas o coordinan los casos de uso.

La independencia no significa que reemplazar una tecnología sea gratuito: una nueva implementación de persistencia debería respetar los contratos requeridos por la aplicación y reproducir garantías como atomicidad, unicidad y control de concurrencia. El objetivo es que ese cambio permanezca contenido en los componentes responsables de la infraestructura y no se propague innecesariamente hacia el dominio.

Uso de IA Generativa y autoría

Durante el desarrollo se utilizaron herramientas de Inteligencia Artificial Generativa como apoyo para análisis, implementación, pruebas y refactoring. Las principales herramientas utilizadas fueron Claude Code y Codex, empleadas siempre dentro del contexto y las restricciones definidas para el obligatorio. Los modelos utilizados fueron Opus 5.5 para el caso de Claude Code y GPT Sol 6.1 para el caso de Codex.

Documentación del uso de IA

La IA se utilizó principalmente para explorar alternativas de diseño, asistir en ciclos TDD, proponer casos de prueba, revisar posibles casos borde, apoyar refactorings y contrastar la solución con la letra, la especificación de la API y las aclaraciones docentes.

Su participación fue especialmente relevante en la revisión de decisiones relacionadas con autenticación y autorización, separación entre componentes, inyección de dependencias, estrategias de ELO y emparejamiento, persistencia, atomicidad y concurrencia.

Las herramientas trabajaron con contexto específico del proyecto, incluyendo la letra vigente, especificación de la API, aclaraciones docentes, archivos AGENTS.md, skills del proyecto, bitácora de decisiones, código y pruebas existentes. Esto permitió mantener las propuestas alineadas con las reglas de la materia y con las decisiones previamente adoptadas.

Las propuestas generadas fueron revisadas antes de incorporarse. Cuando una alternativa no respetaba la arquitectura, las restricciones del obligatorio o los conceptos trabajados en el curso, fue modificada o descartada. La responsabilidad sobre las decisiones, el código y la documentación permaneció en el equipo.

Bitácora de decisiones de diseño

Durante el desarrollo se mantuvo una bitácora con las principales decisiones que tuvieron impacto sobre la arquitectura, mantenibilidad, extensibilidad, testeabilidad o consistencia de la solución.

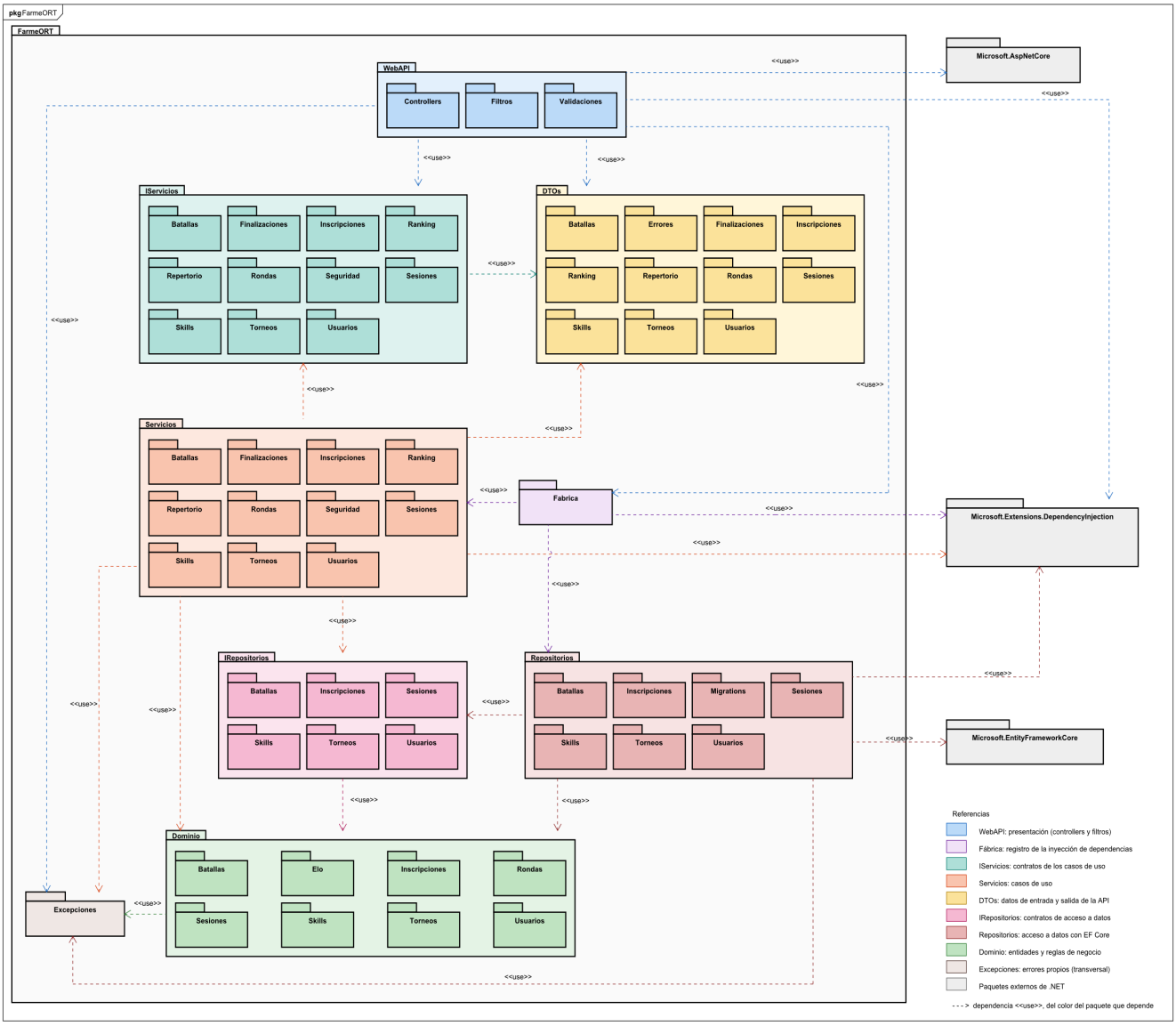
Para cada decisión relevante se registraron el problema, las alternativas consideradas, la decisión adoptada y su justificación, los conceptos del curso utilizados, la evidencia asociada y, cuando correspondió, la participación de IA.

El Anexo F — Bitácora de decisiones de diseño presenta una selección de las decisiones más representativas del diseño final de FarmeORT.

Anexo

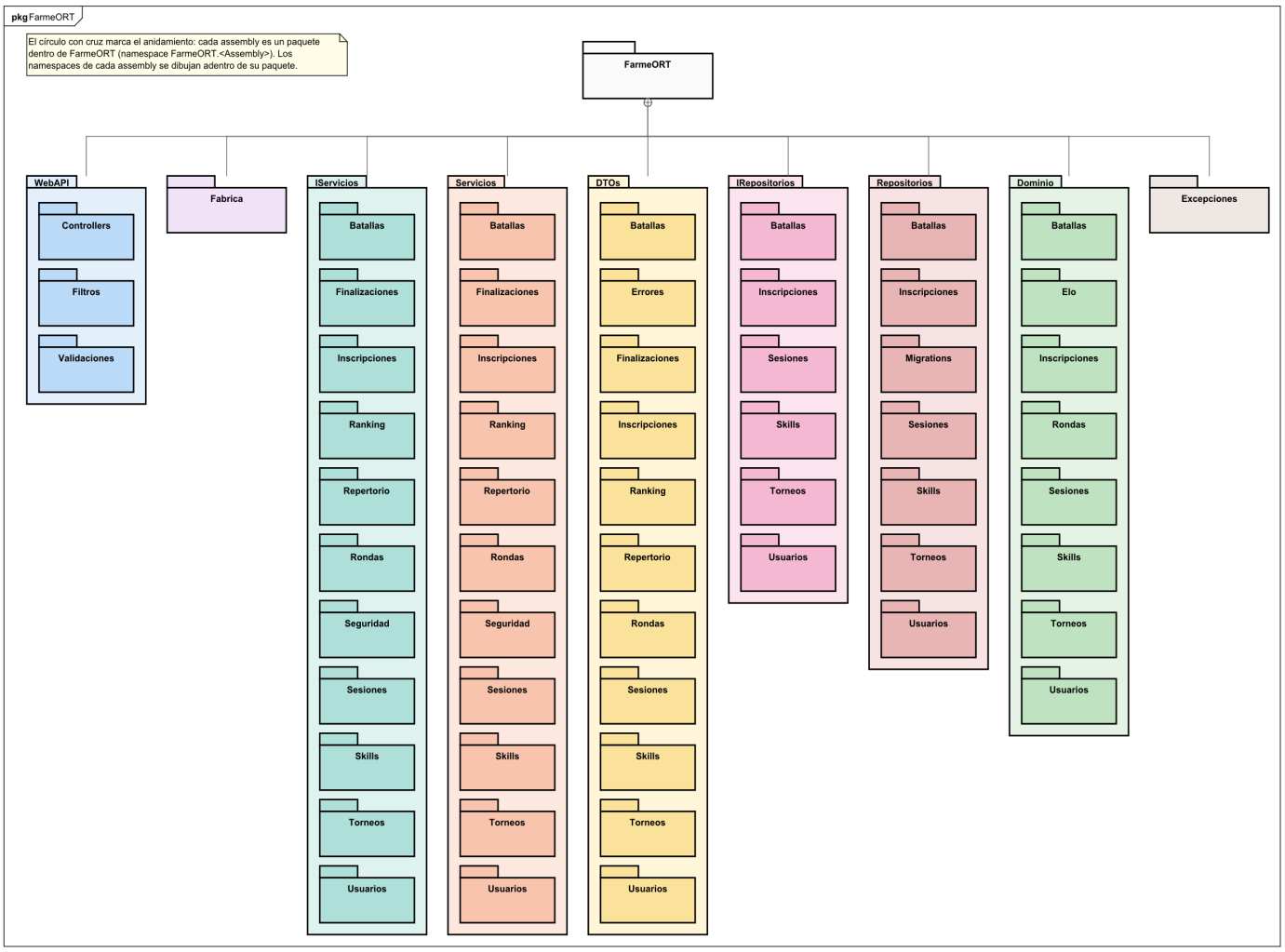
Anexo A.1

Diagrama de paquetes



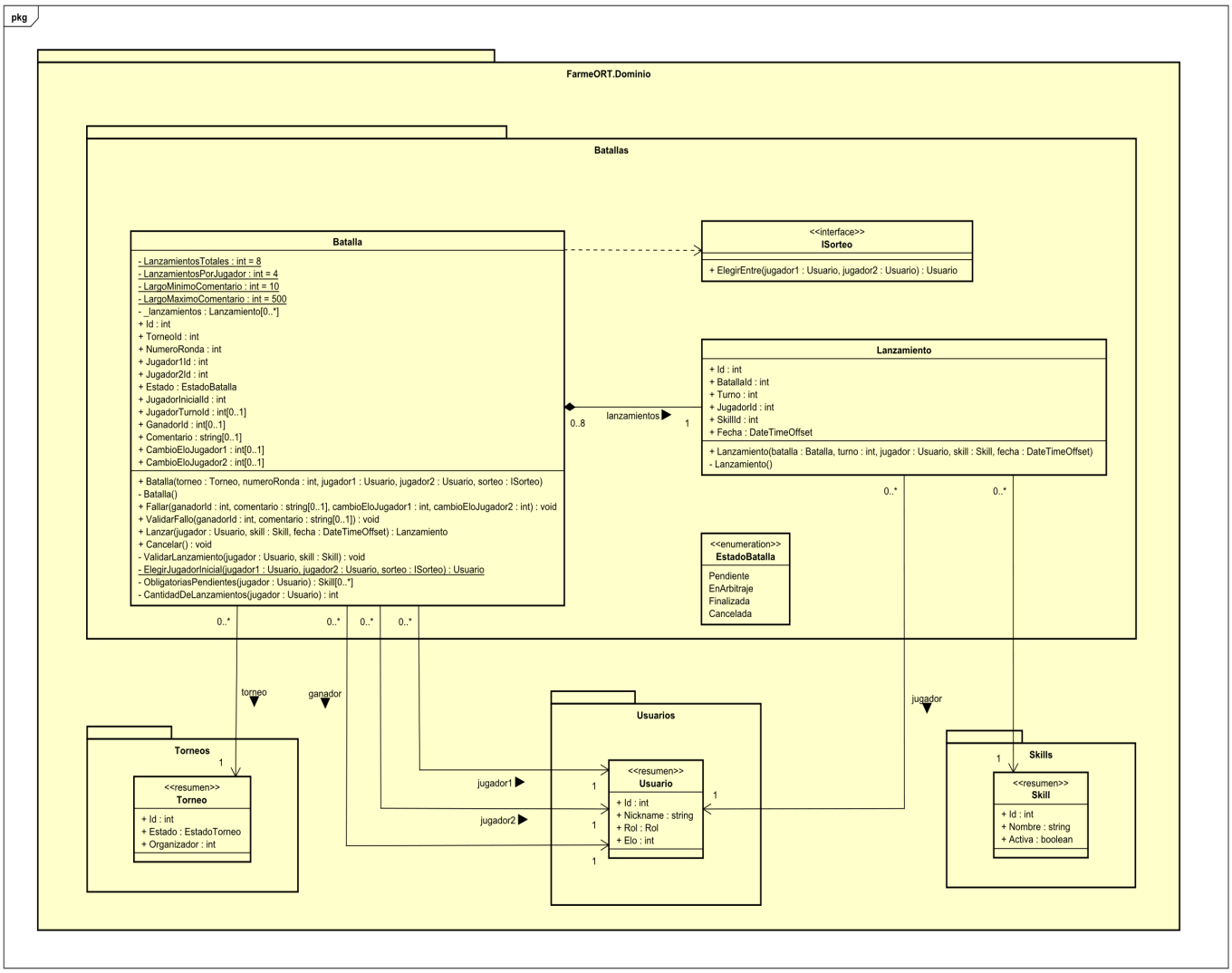
Anexo A.2

Anidamiento de paquetes

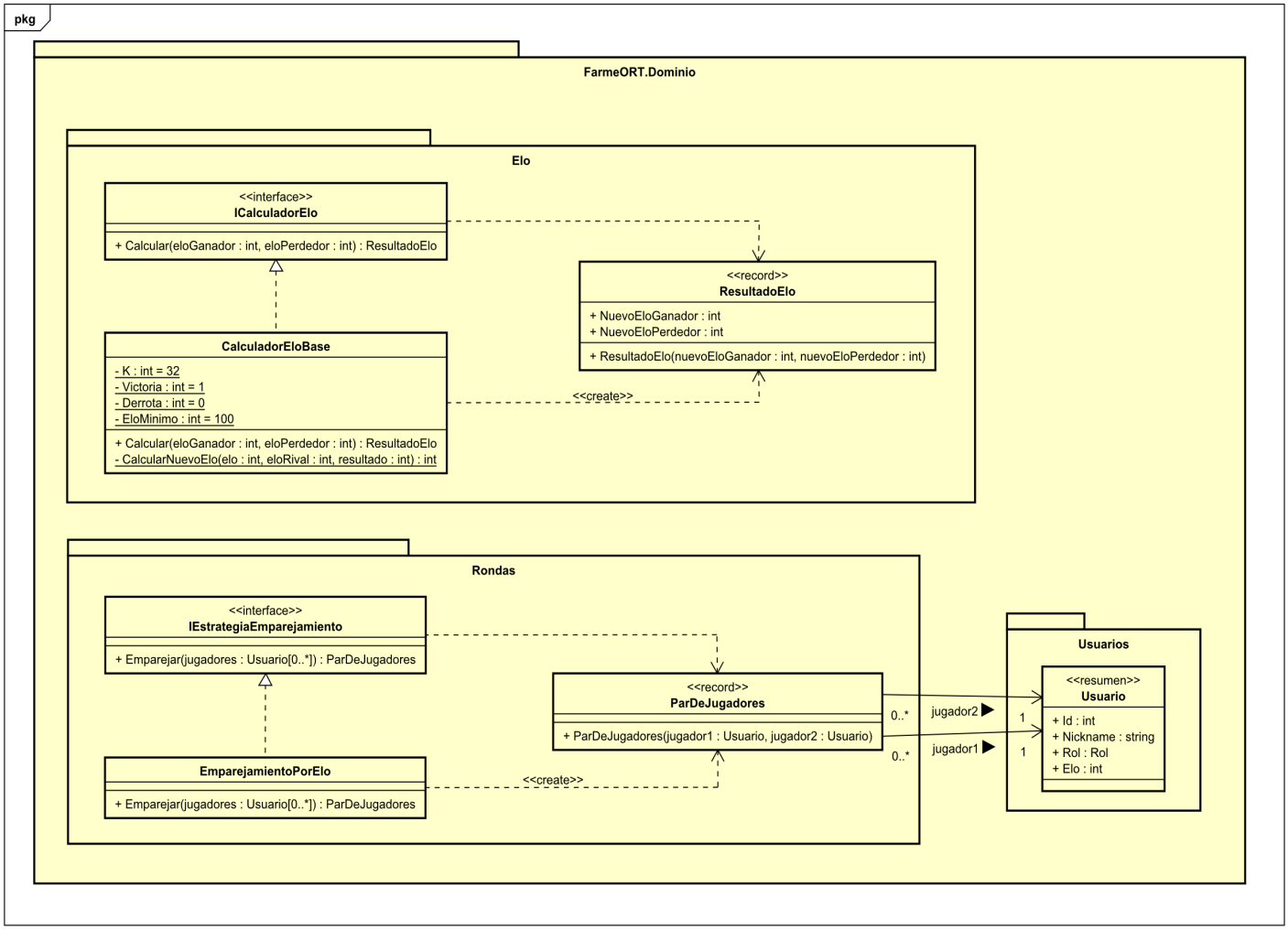


Anexo B.1

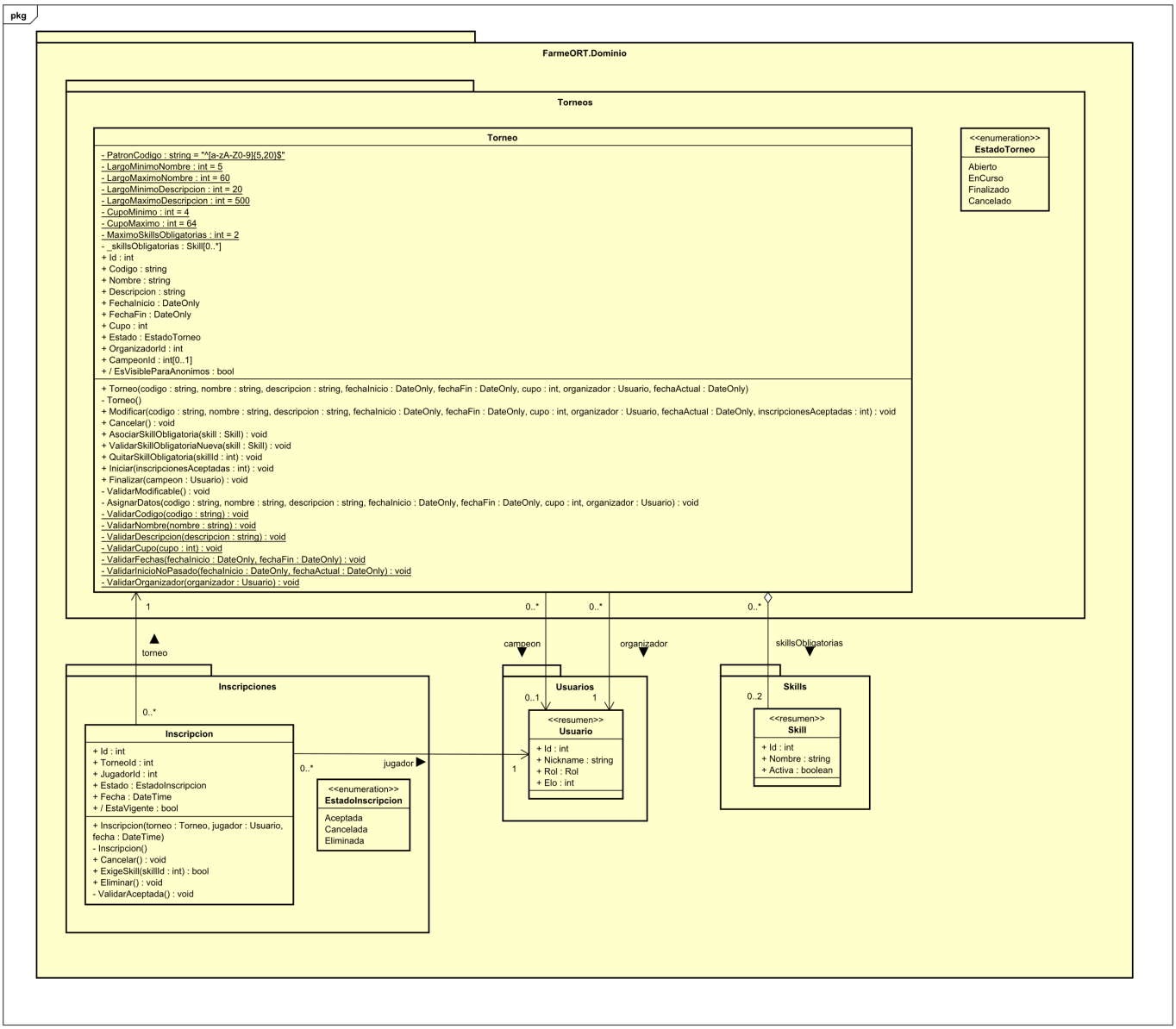
DominioBatallasLanzamientos



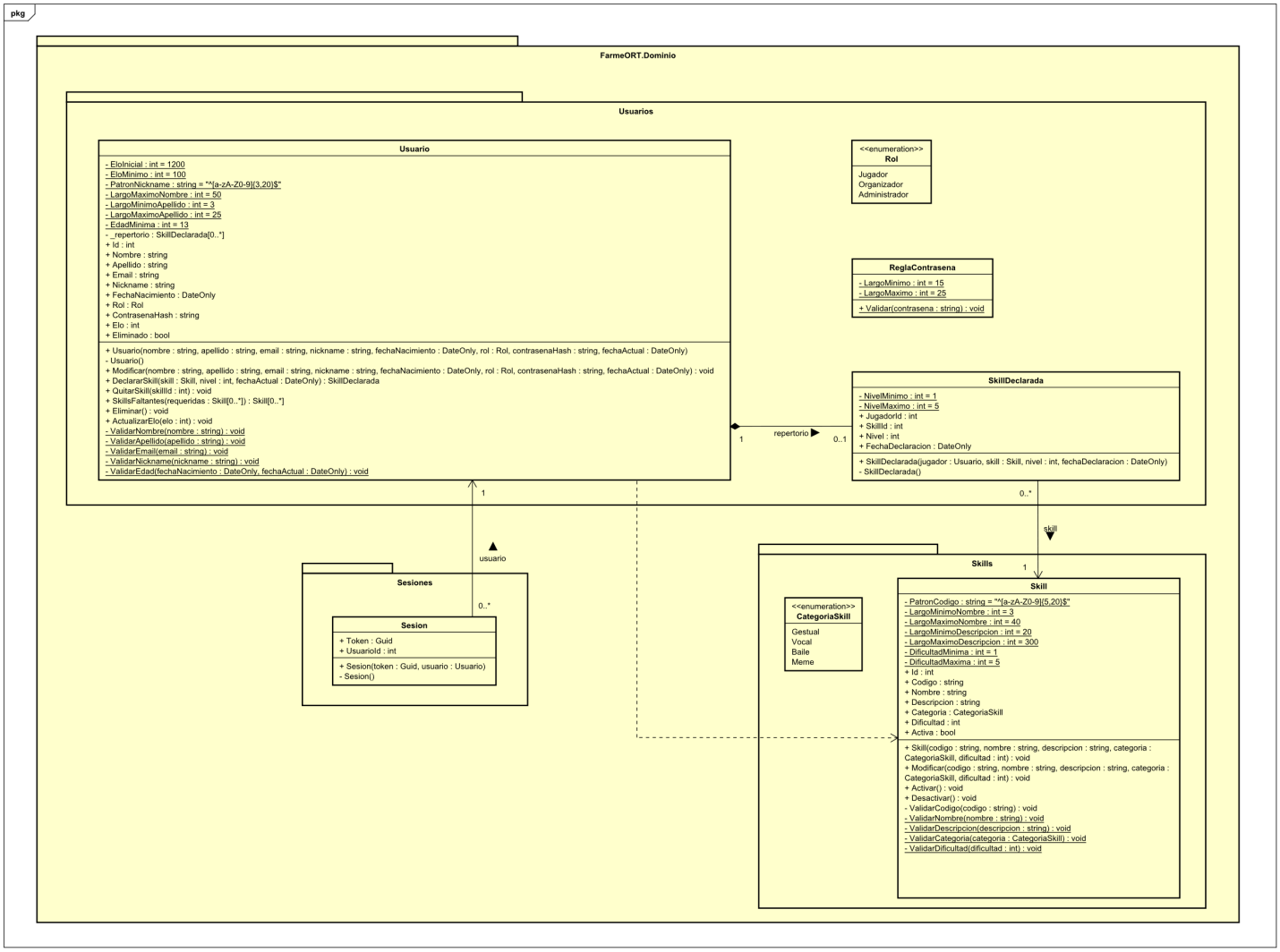
DominioRondasElo



DominioTorneosInscripciones

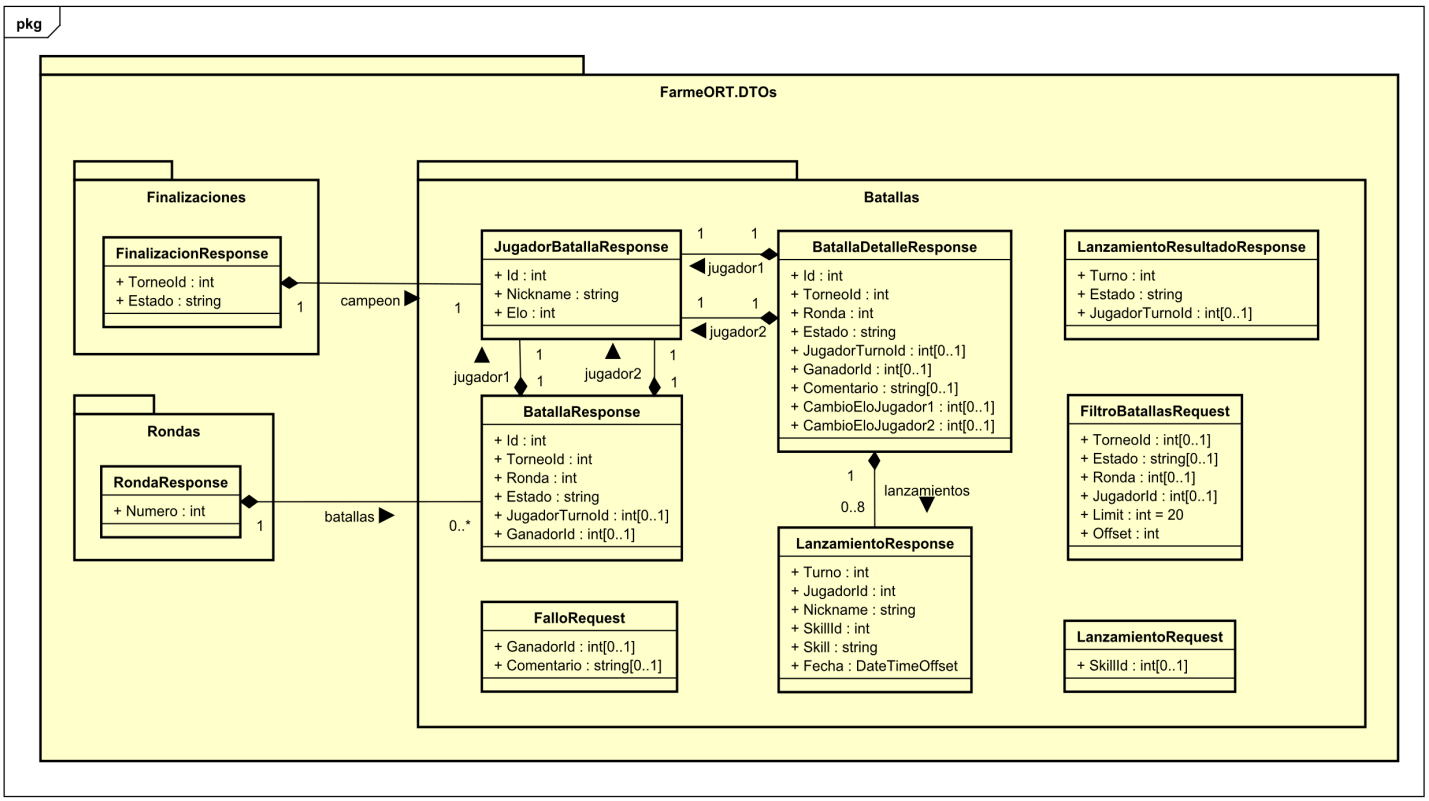


DominioUsuariosSkillsSesiones

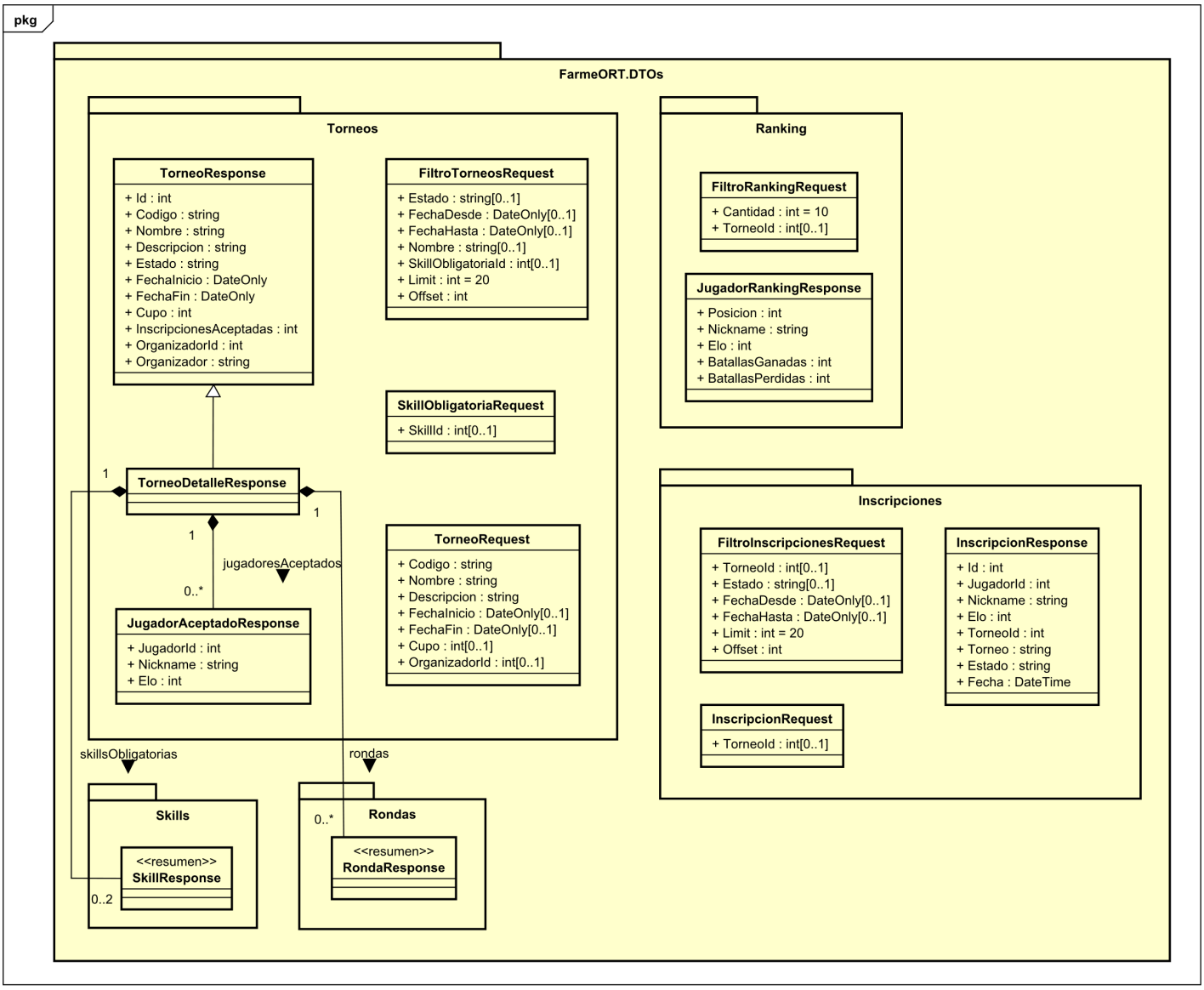


Anexo B.2

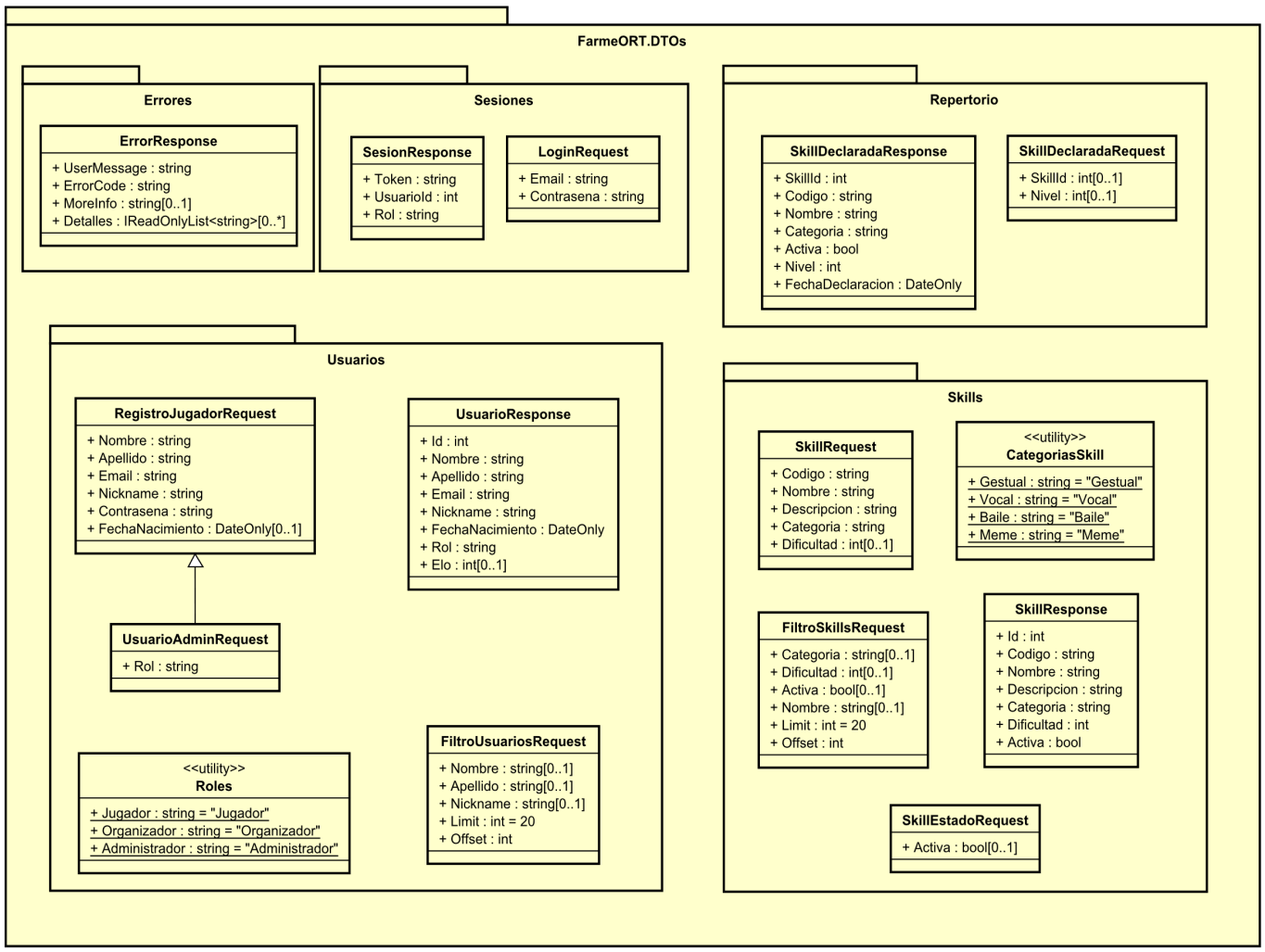
DTOsBatallasLanzamientosFallosRondasFinalizacion



DTOsTorneosInscripcionesRanking

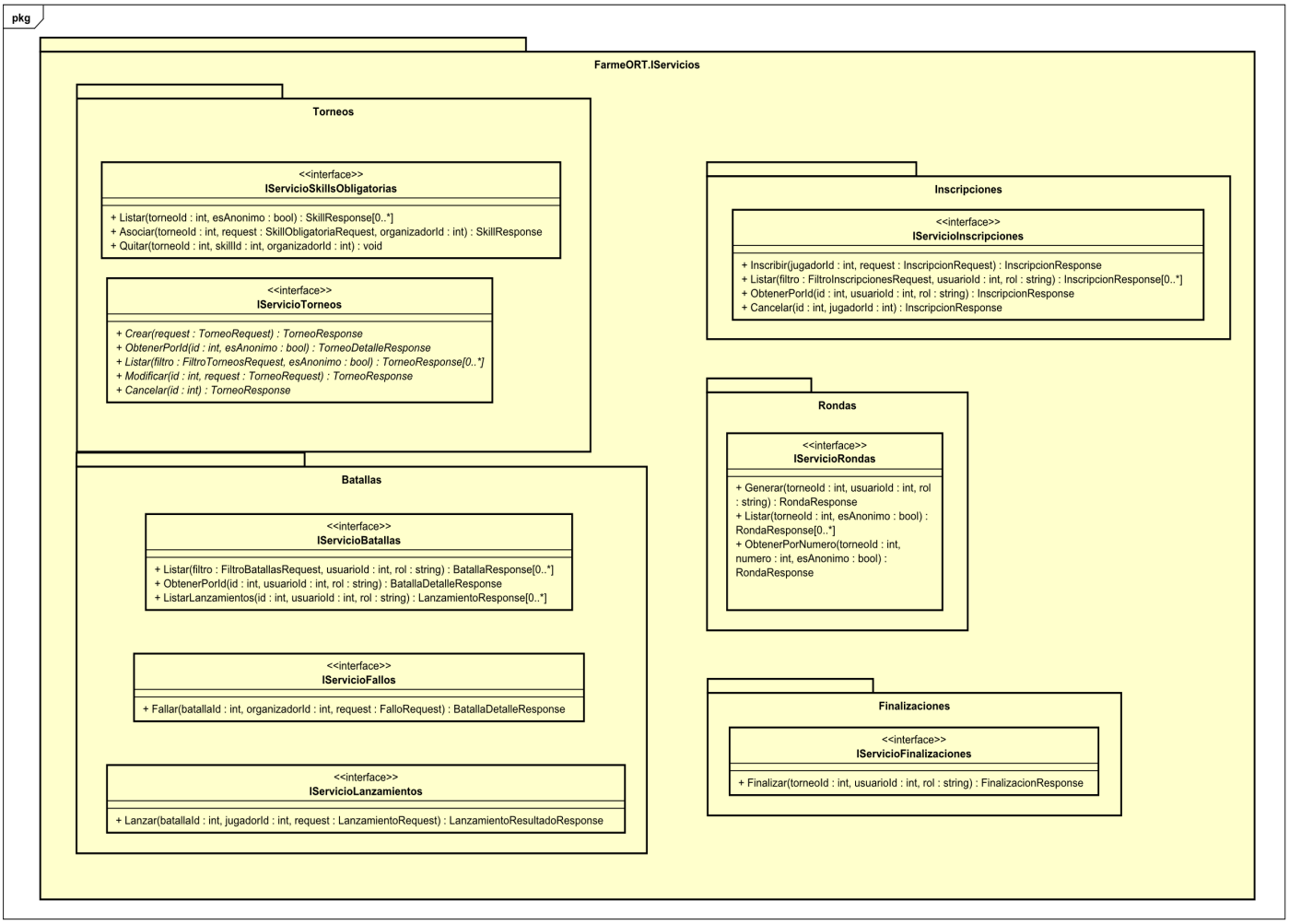


DTOsUsuariosSesionesErroresSkillsRepertorios

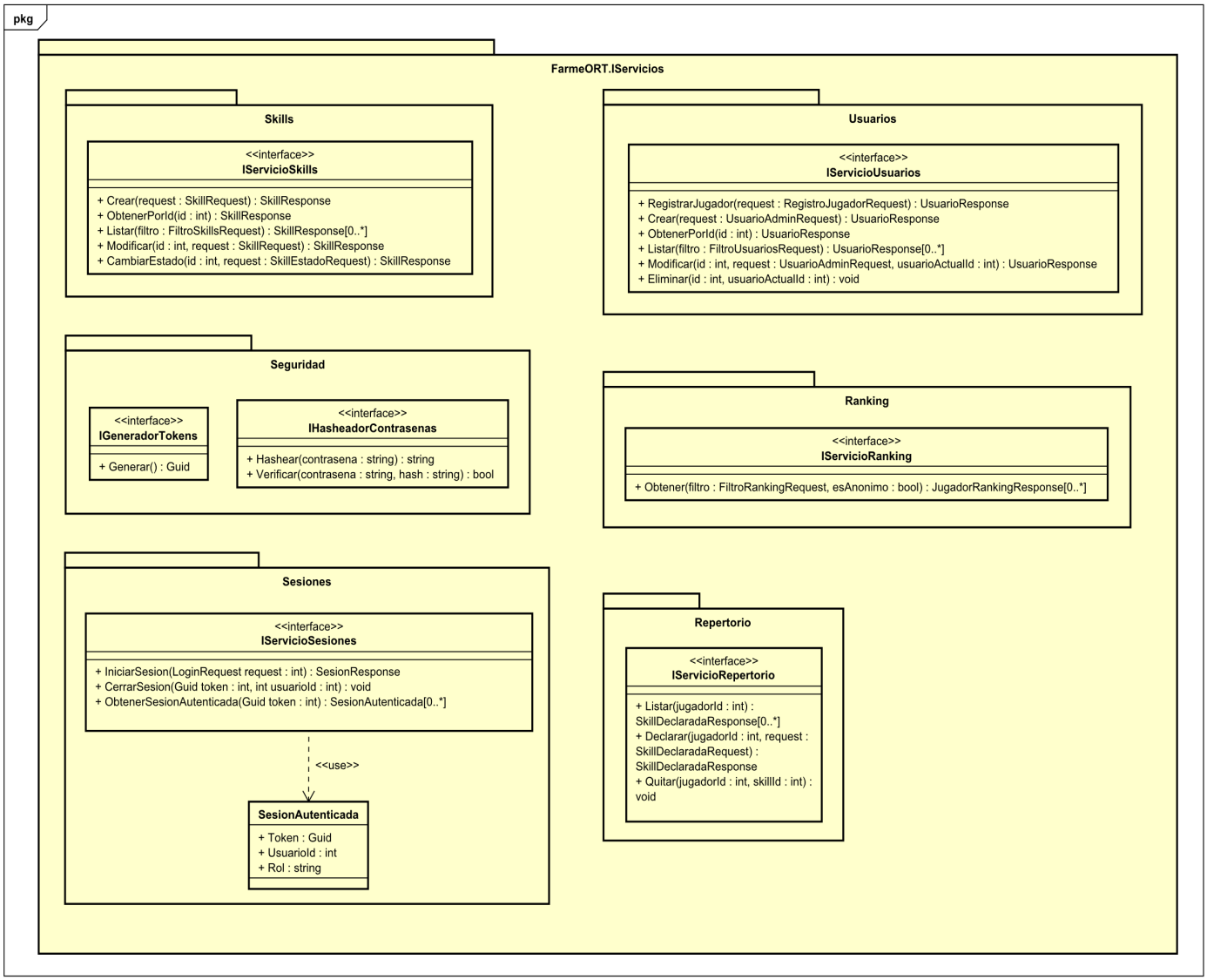


Anexo B.3

IServiciosTorneosInscripcionesBatallasRondasFinalizacion

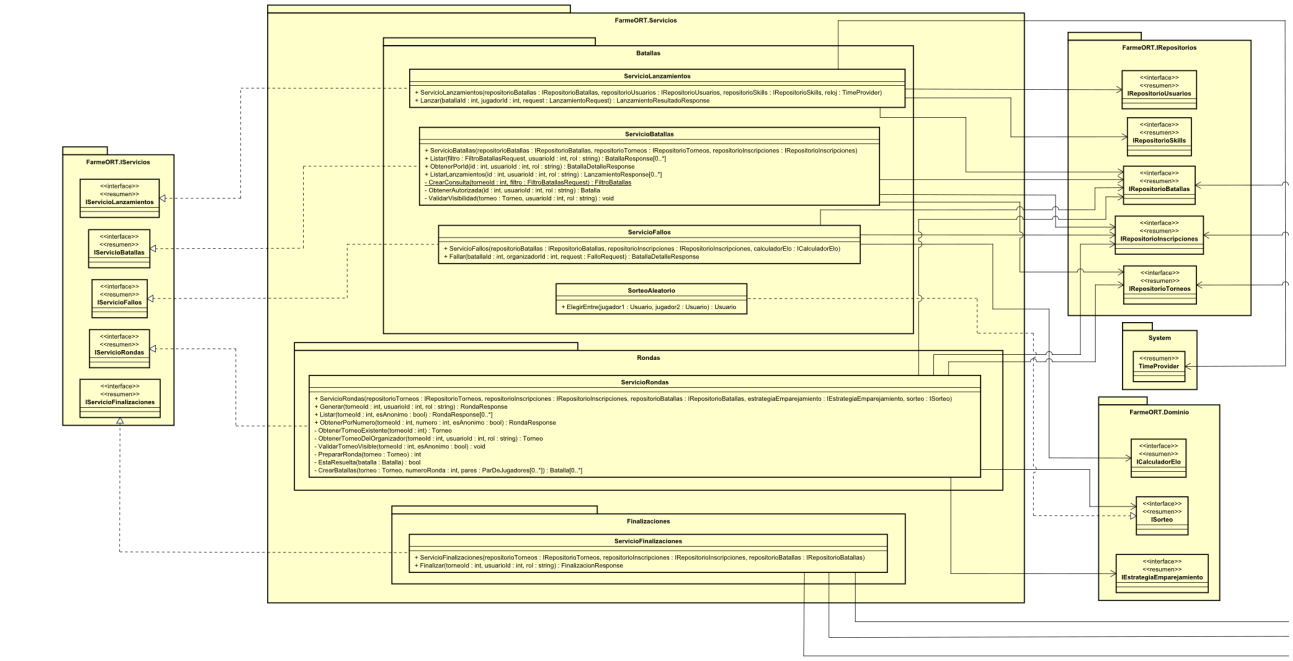


IServiciosUsuariosSesionesSeguridadSkillsReportorioRanking

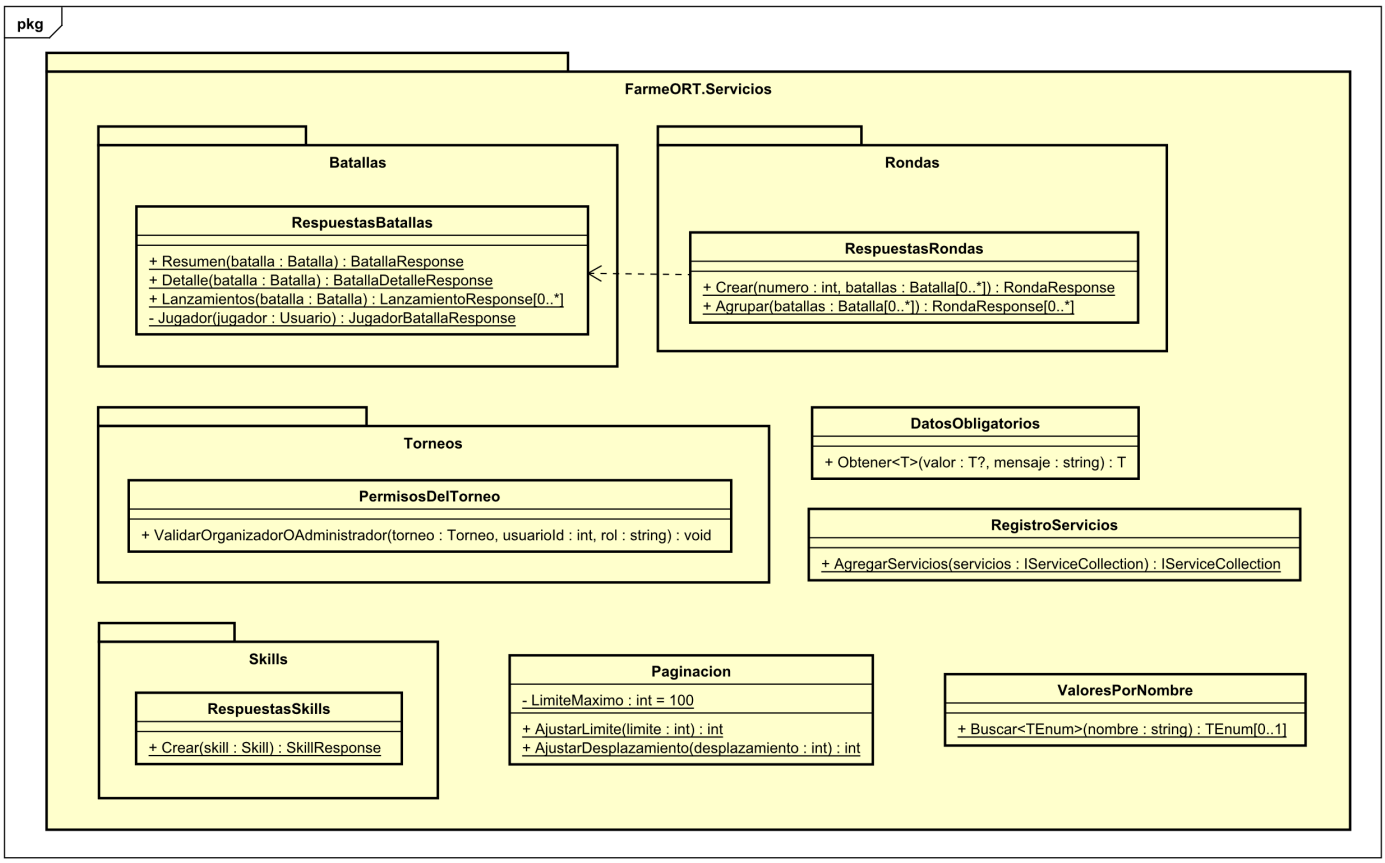


Anexo B.4

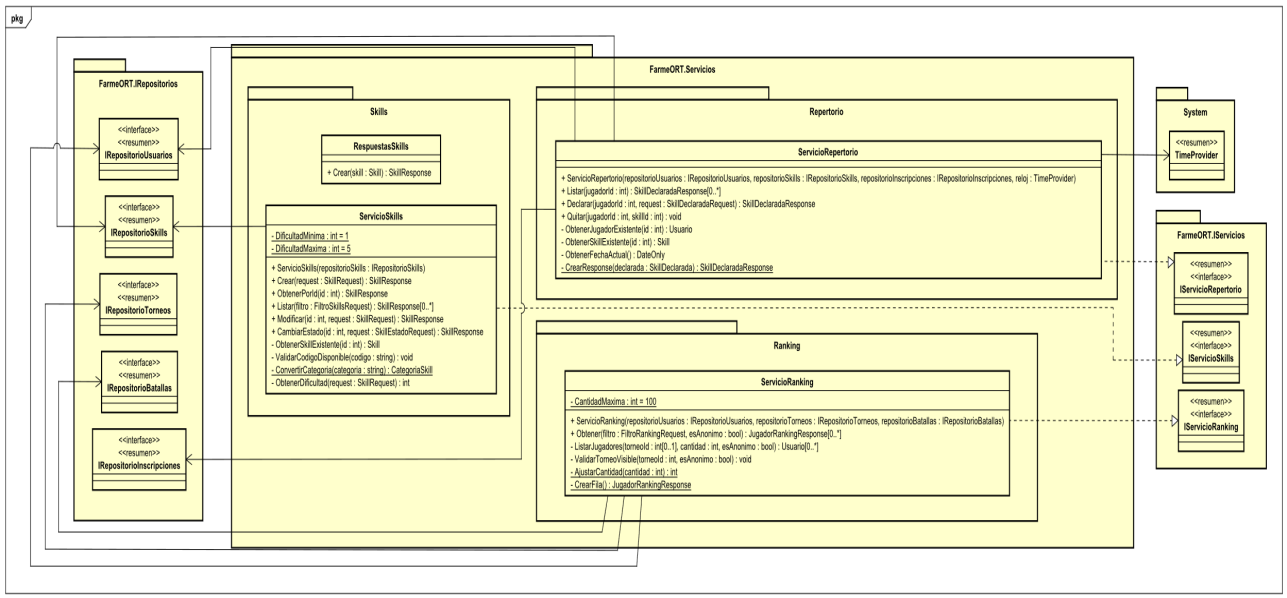
ServiciosBatallasLanzamientosFallosRondasFinalizacion



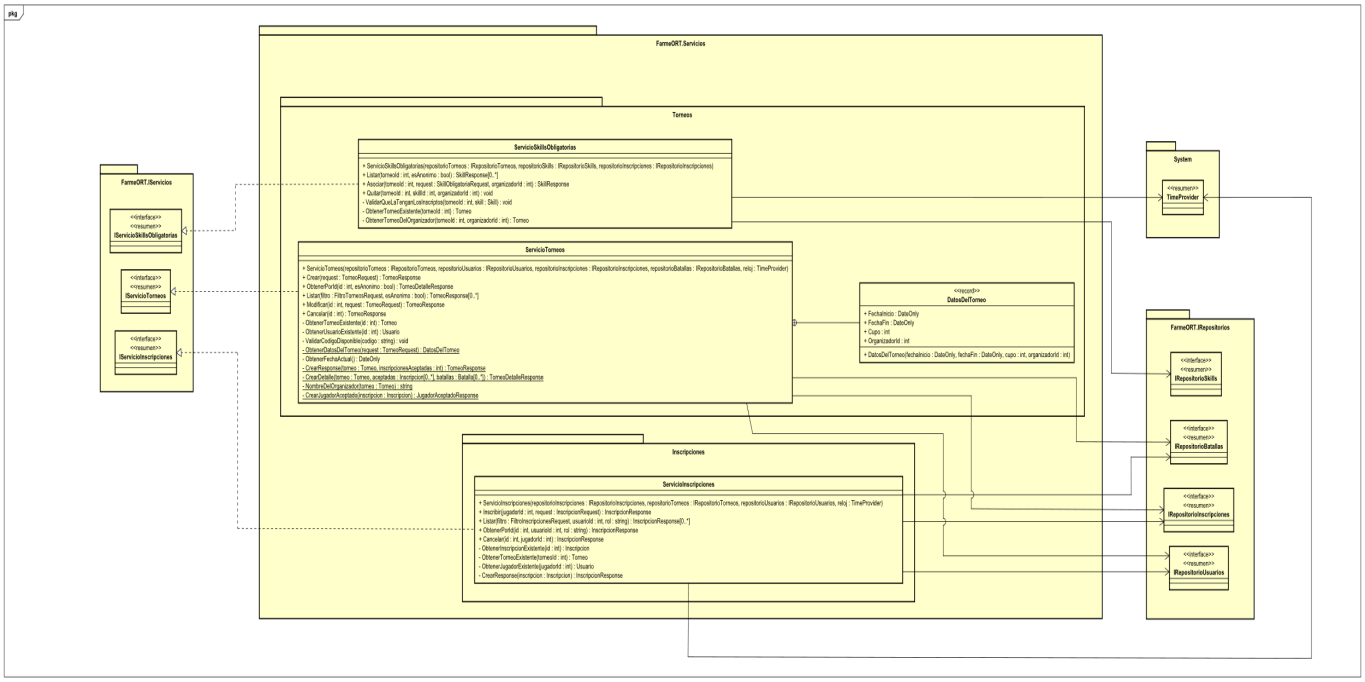
ServiciosClasesAuxiliaresRegistroDeDependencias



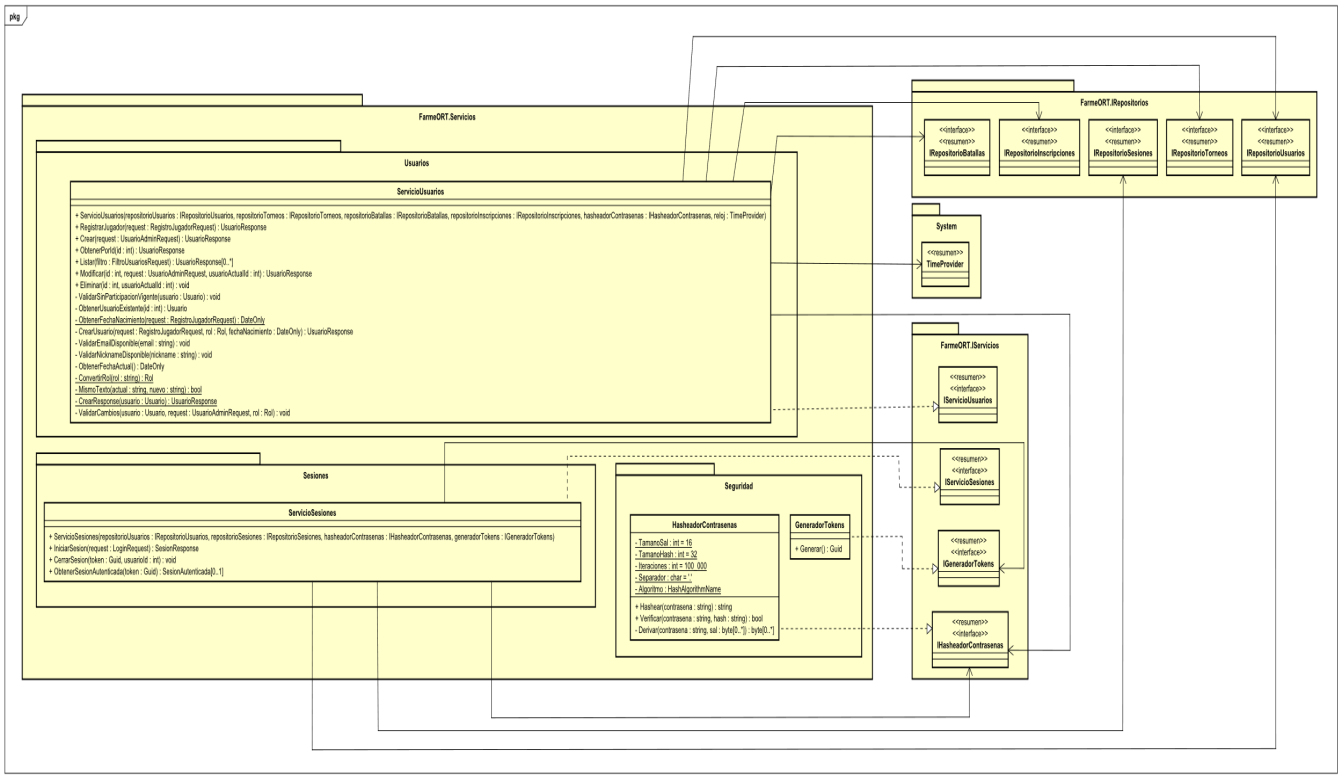
ServiciosSkillsRepertorioRanking



ServiciosTorneosSkillsObligatoriasInscripciones

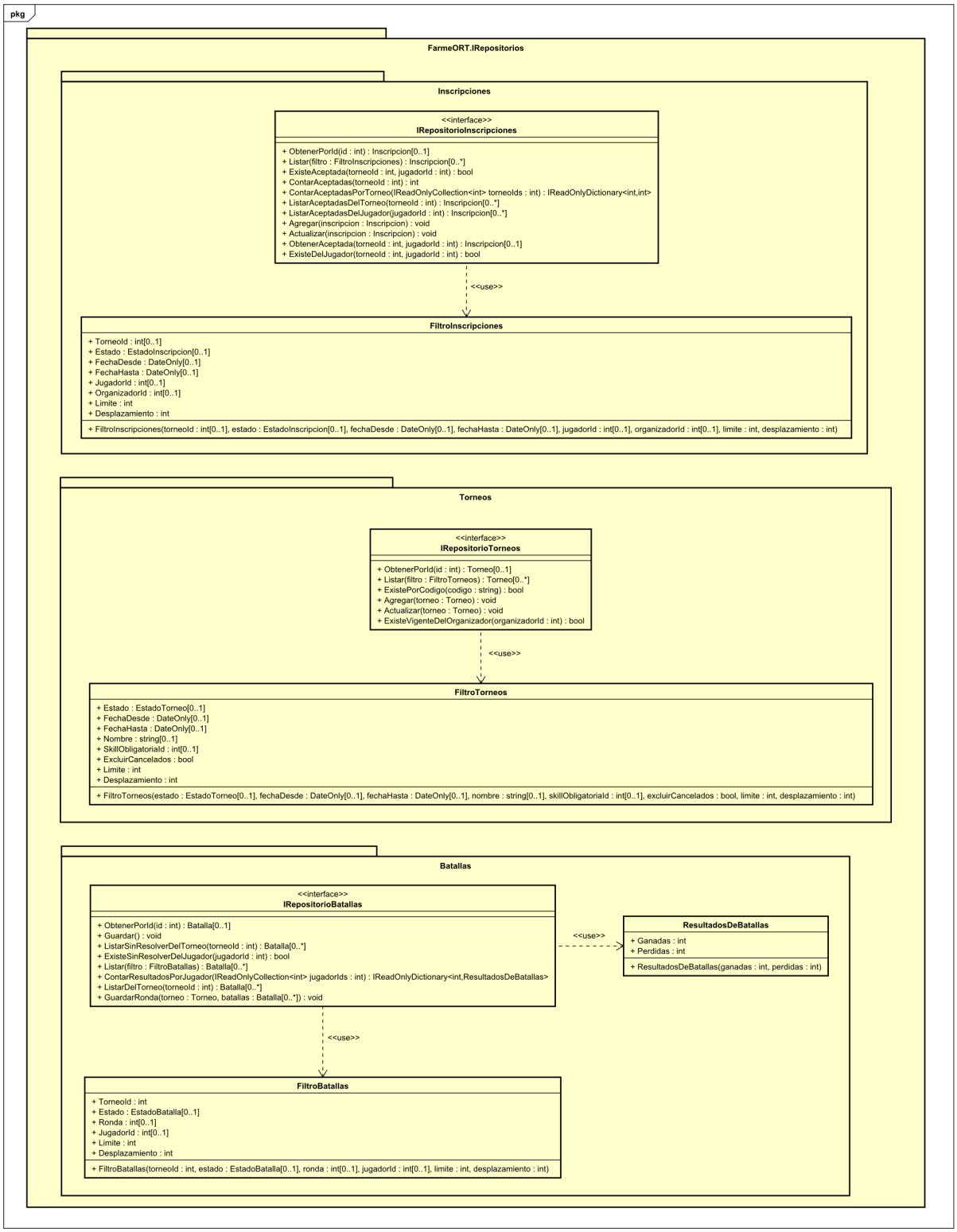


ServiciosUsuariosSesionesSeguridad

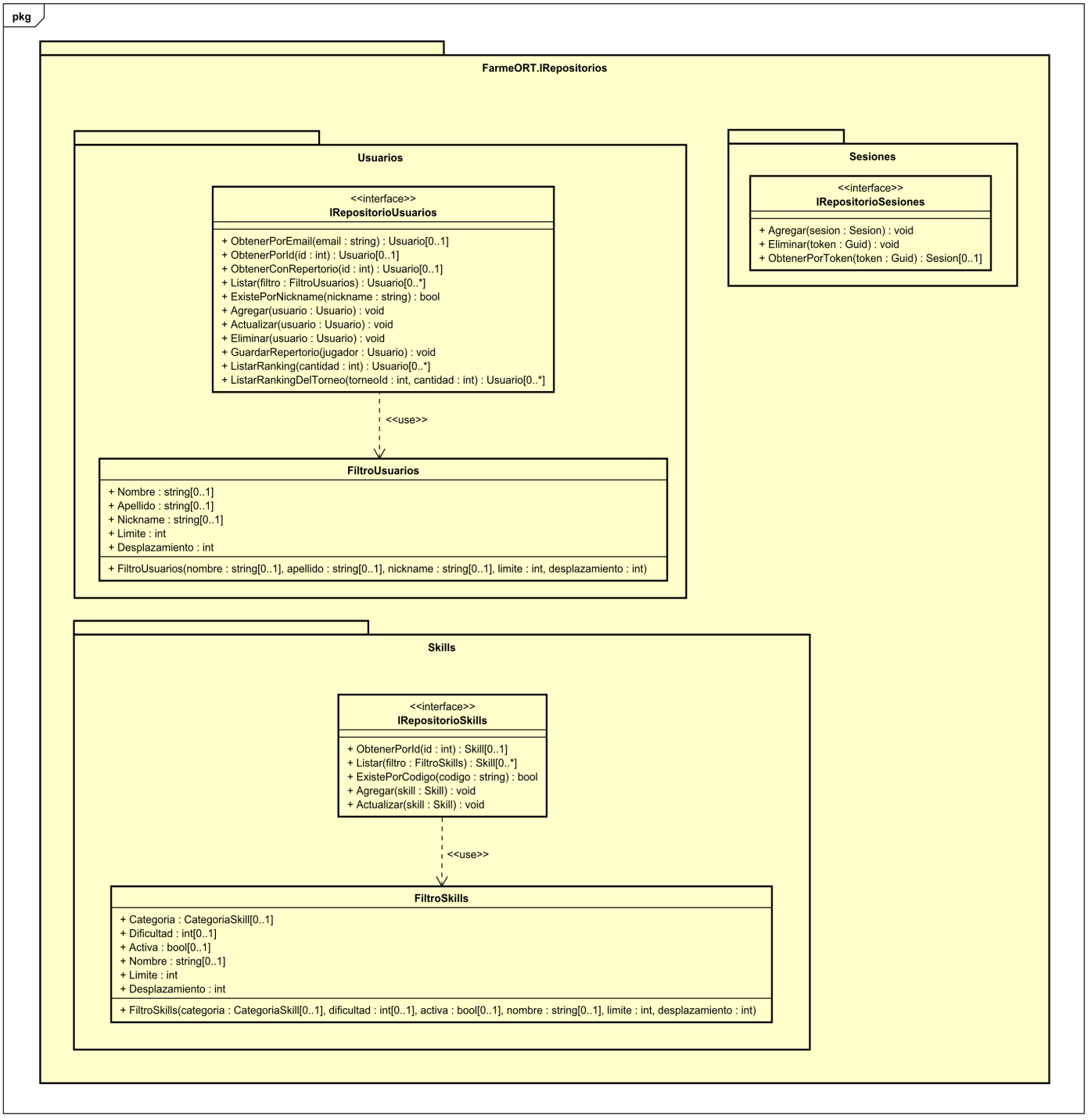


Anexo B.5

IRepositoriosTorneosInscripcionesBatallas

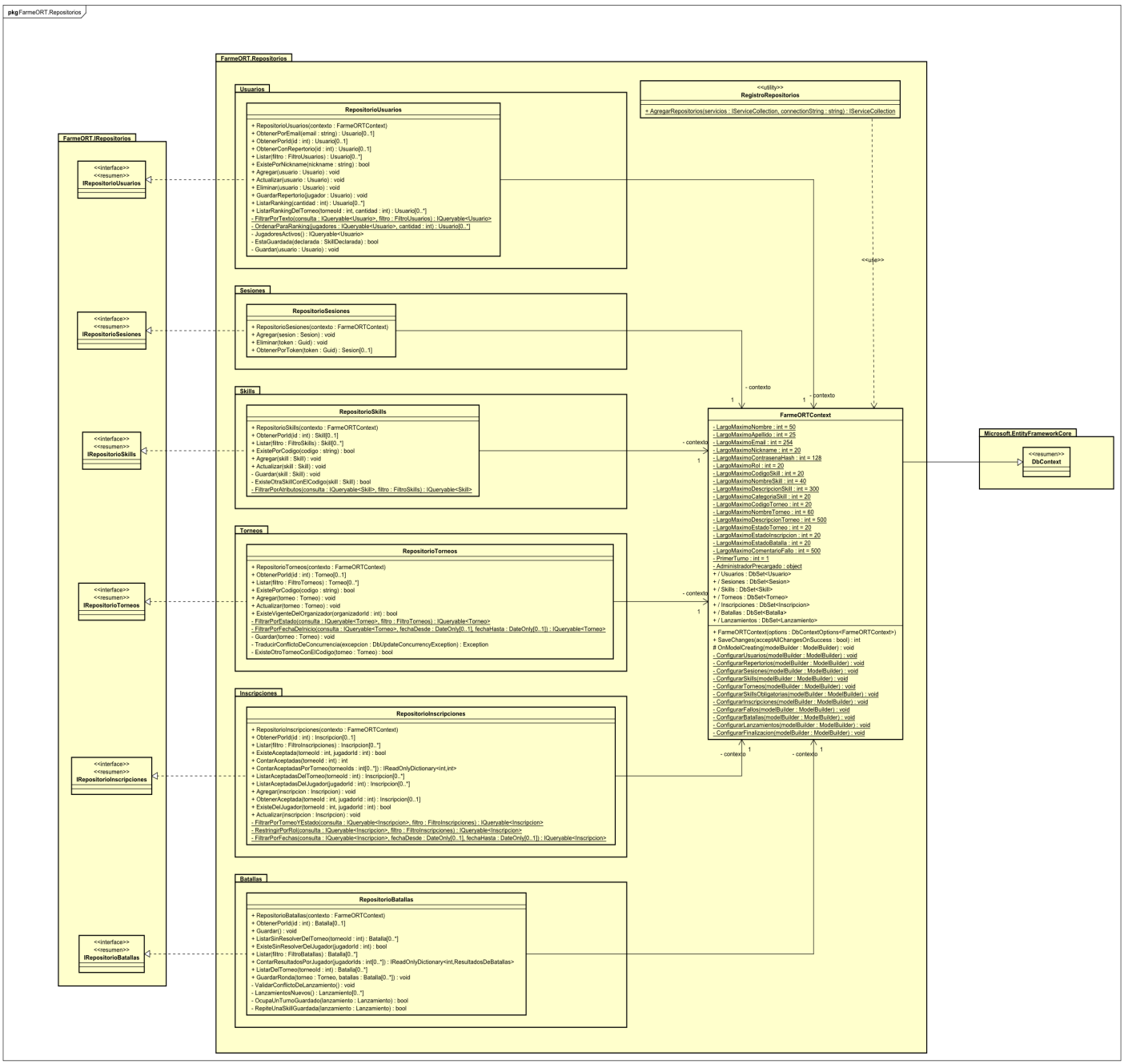


IRepositoriosUsuariosSesionesSkills



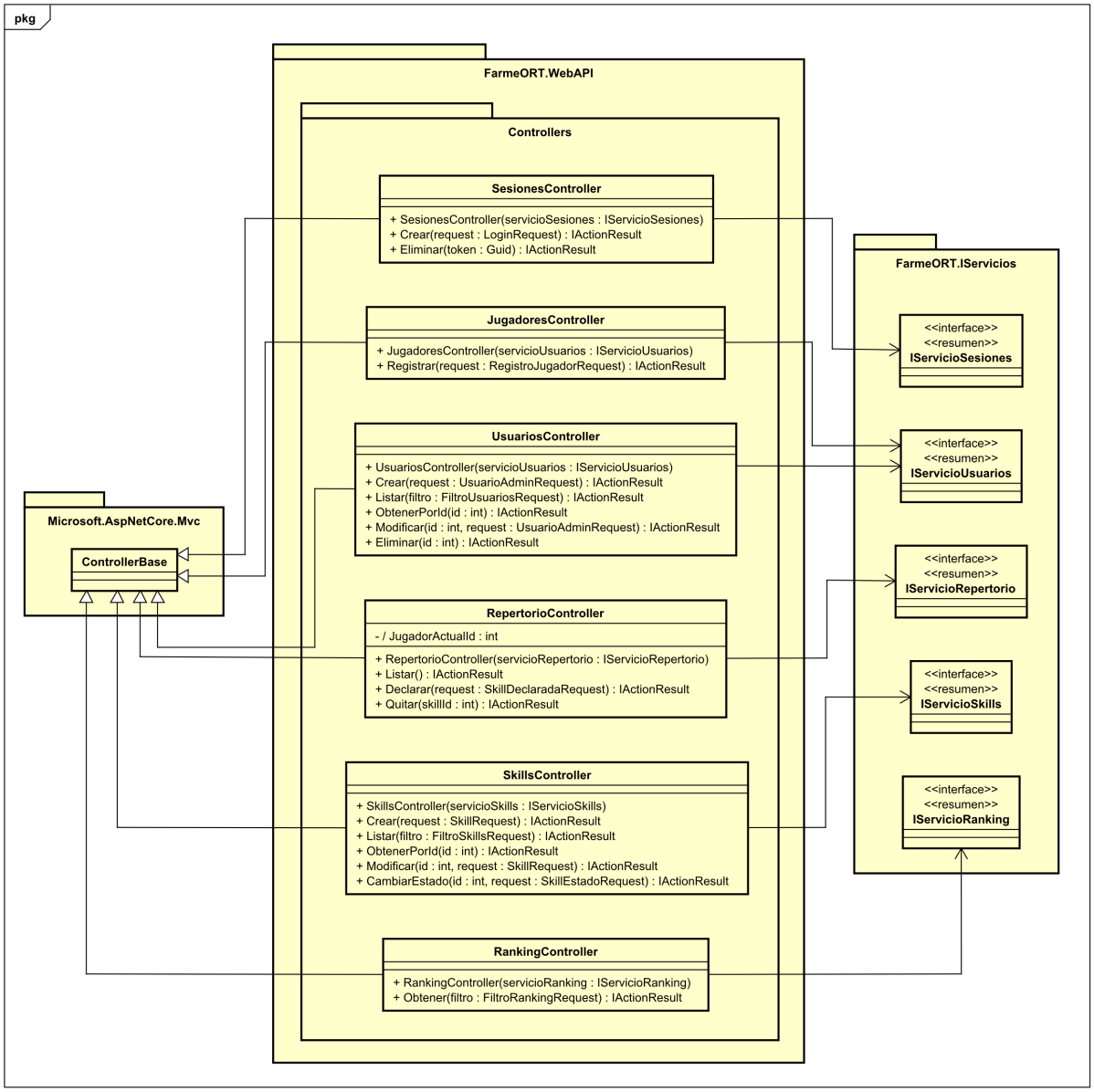
Anexo B.6

Diagrama de clases de Repositorios

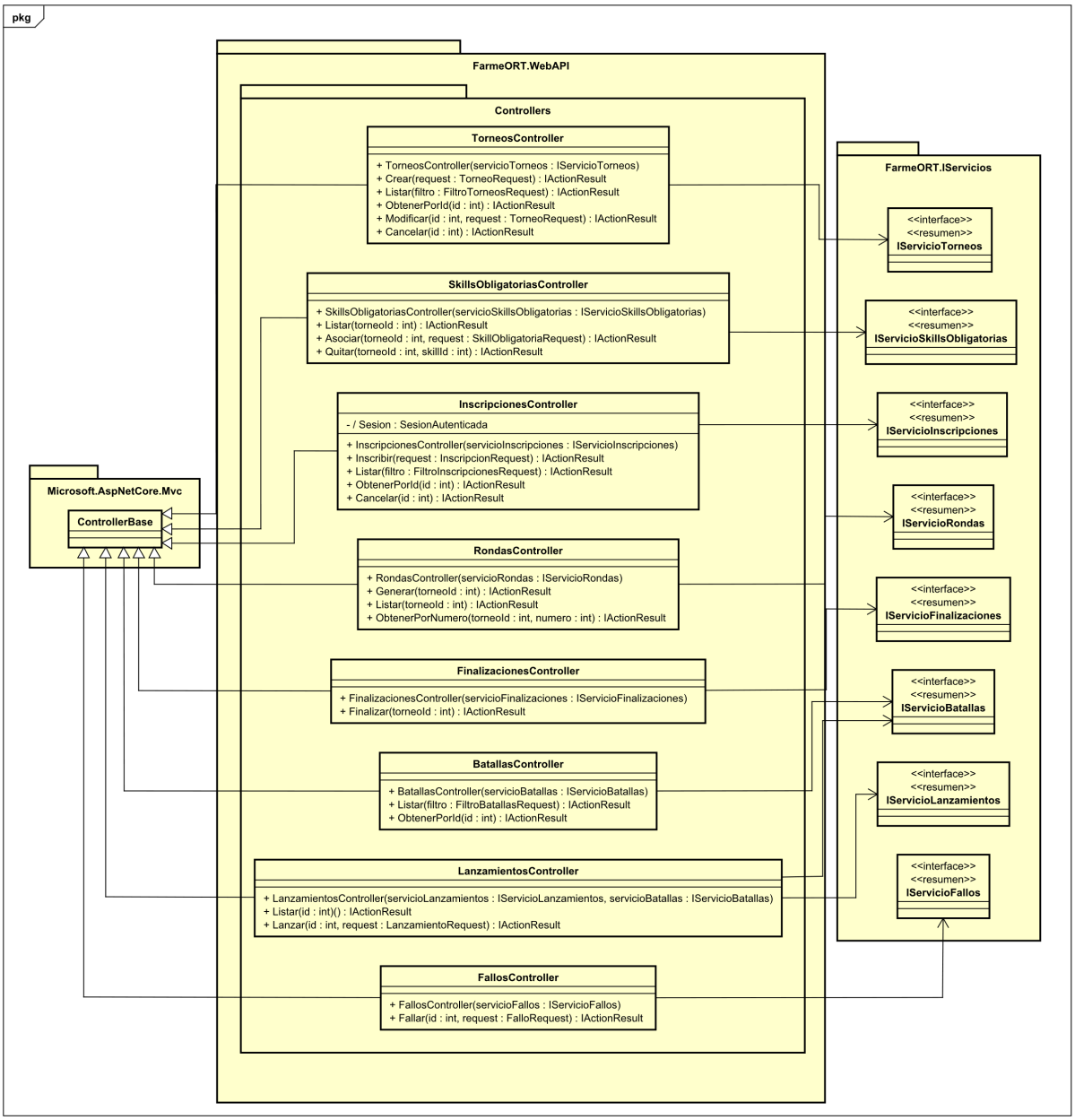


Anexo B.7

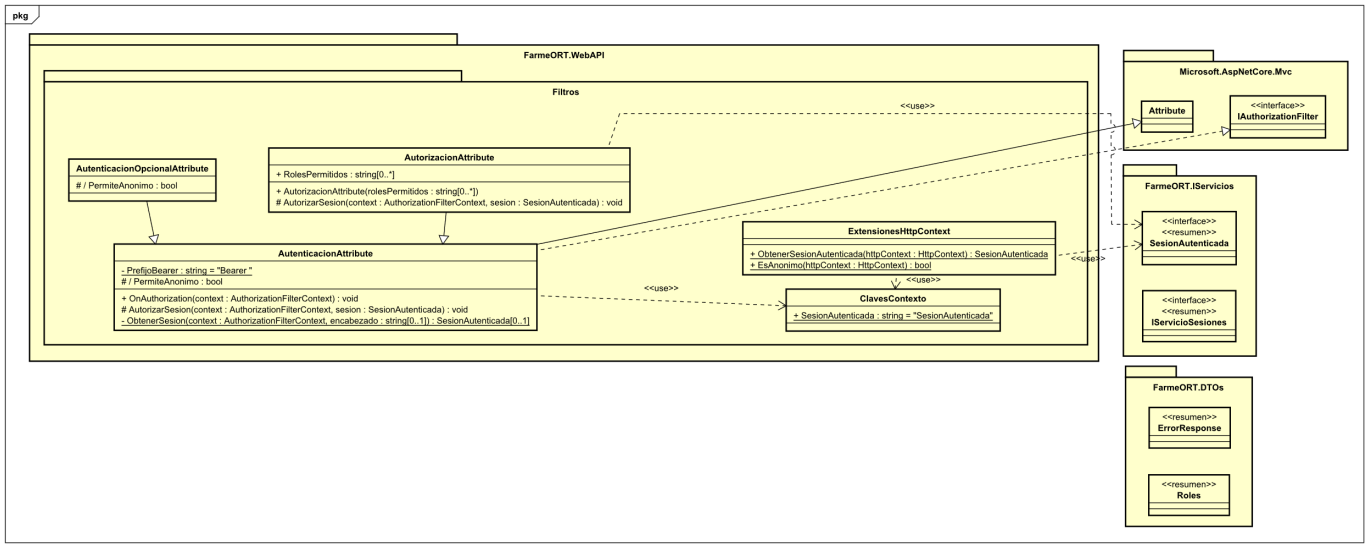
WebAPIControllersSesionesUsuariosRepertorioSkillsRanking



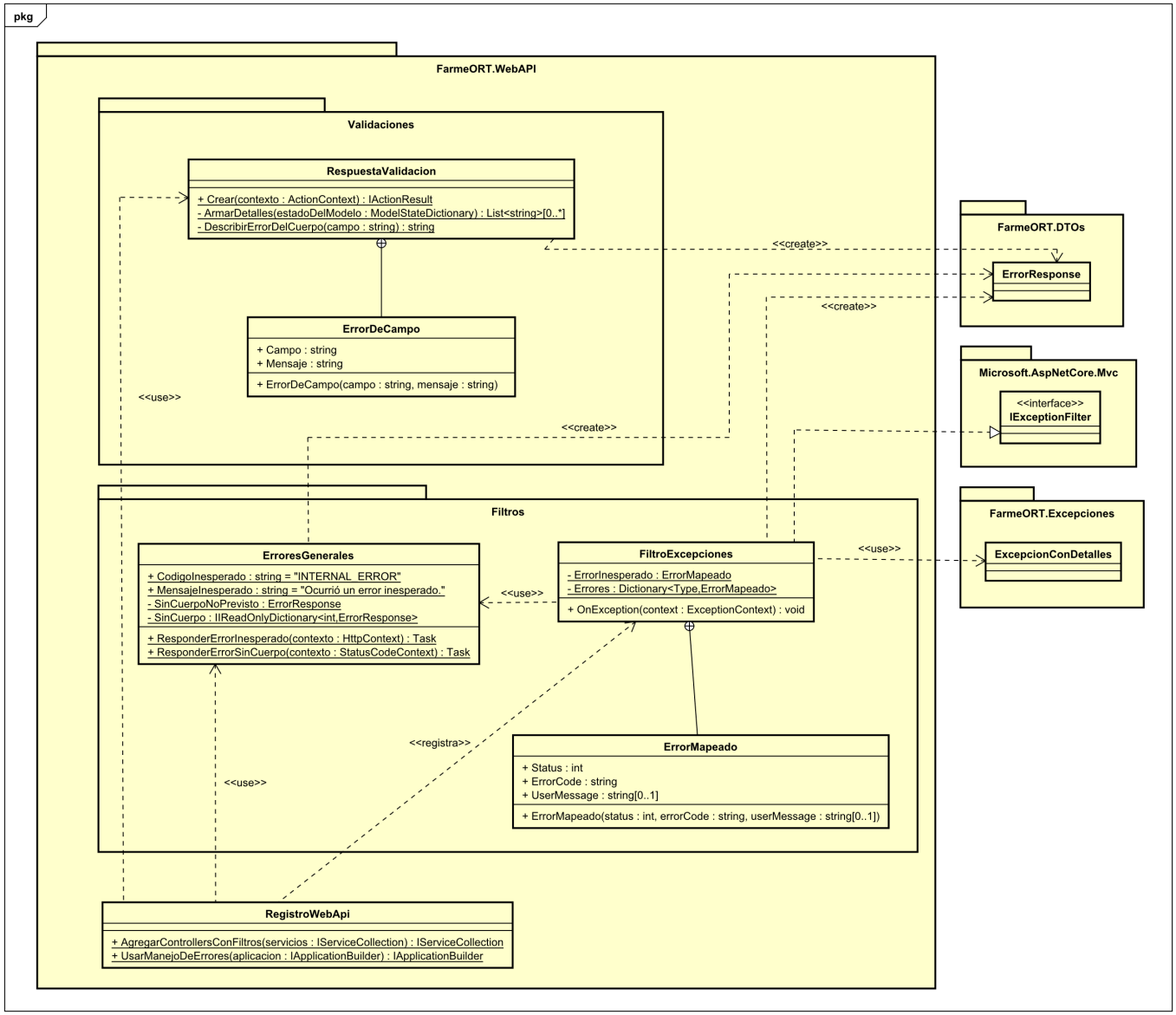
WebAPIControllersTorneosInscripcionesRondasBatallas



WebAPIFiltrosDeAutenticacionYAutorizacion

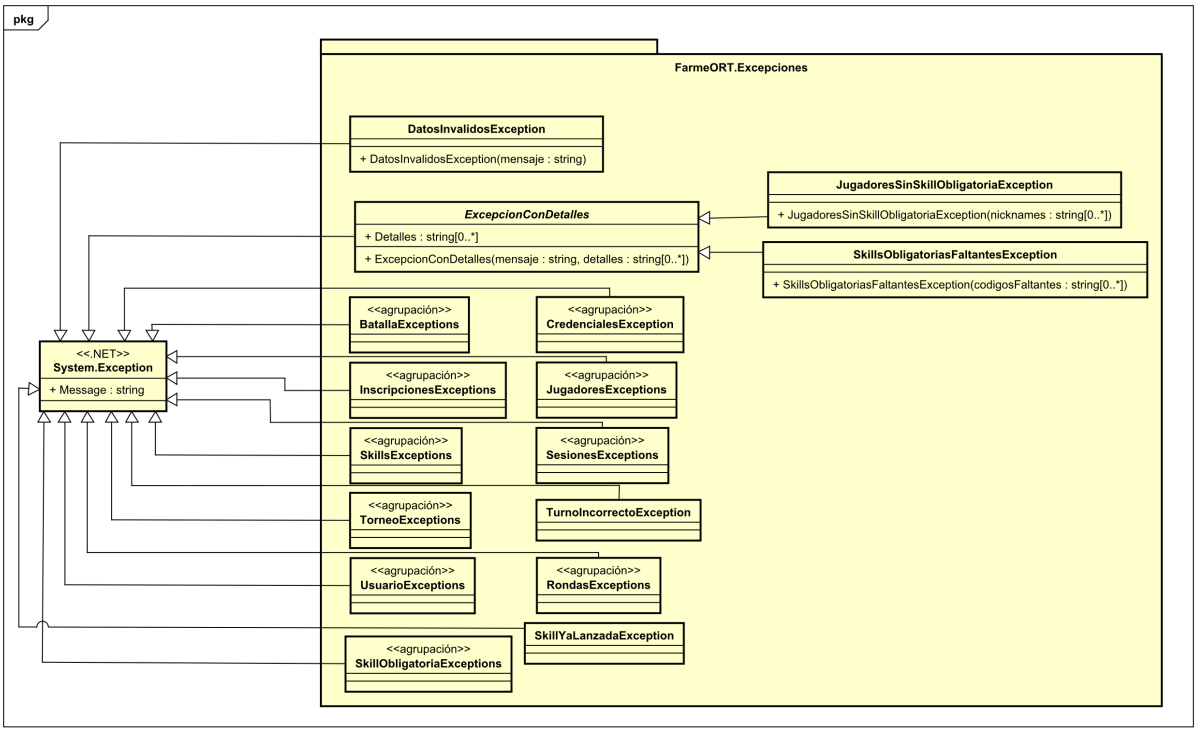


WebAPIManejoDeErroresValidacionRegistro



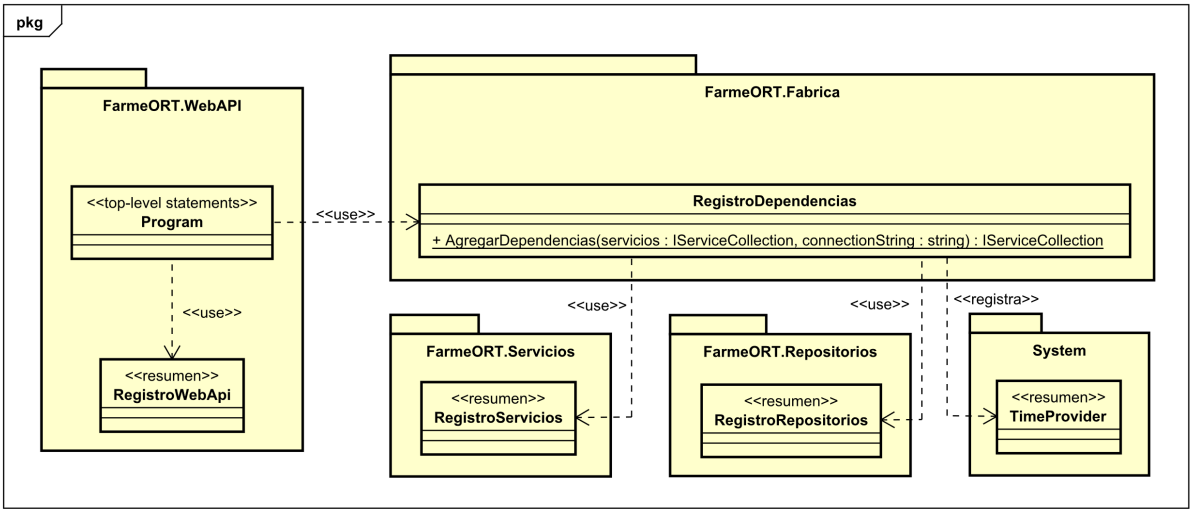
Anexo B.8

ExcepcionesJerarquiaErrores



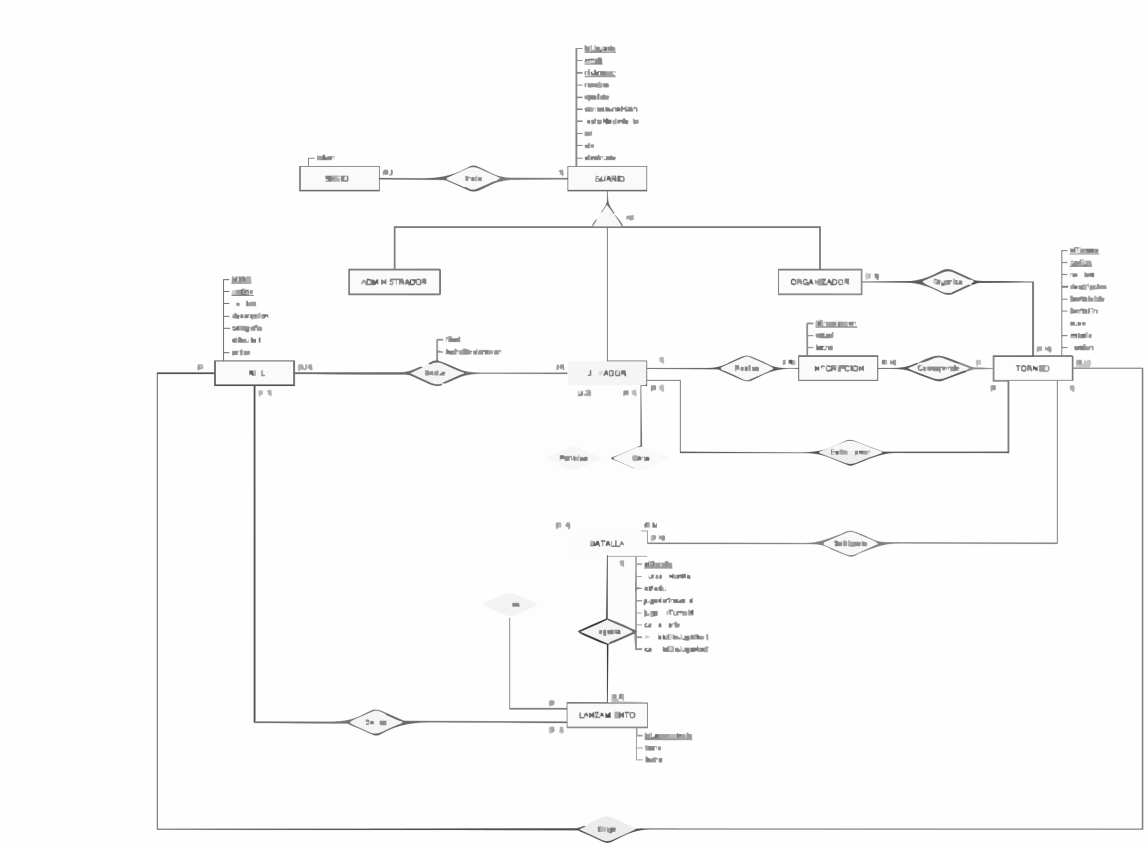
Anexo B.9

FabricaRegistroDeDependencias



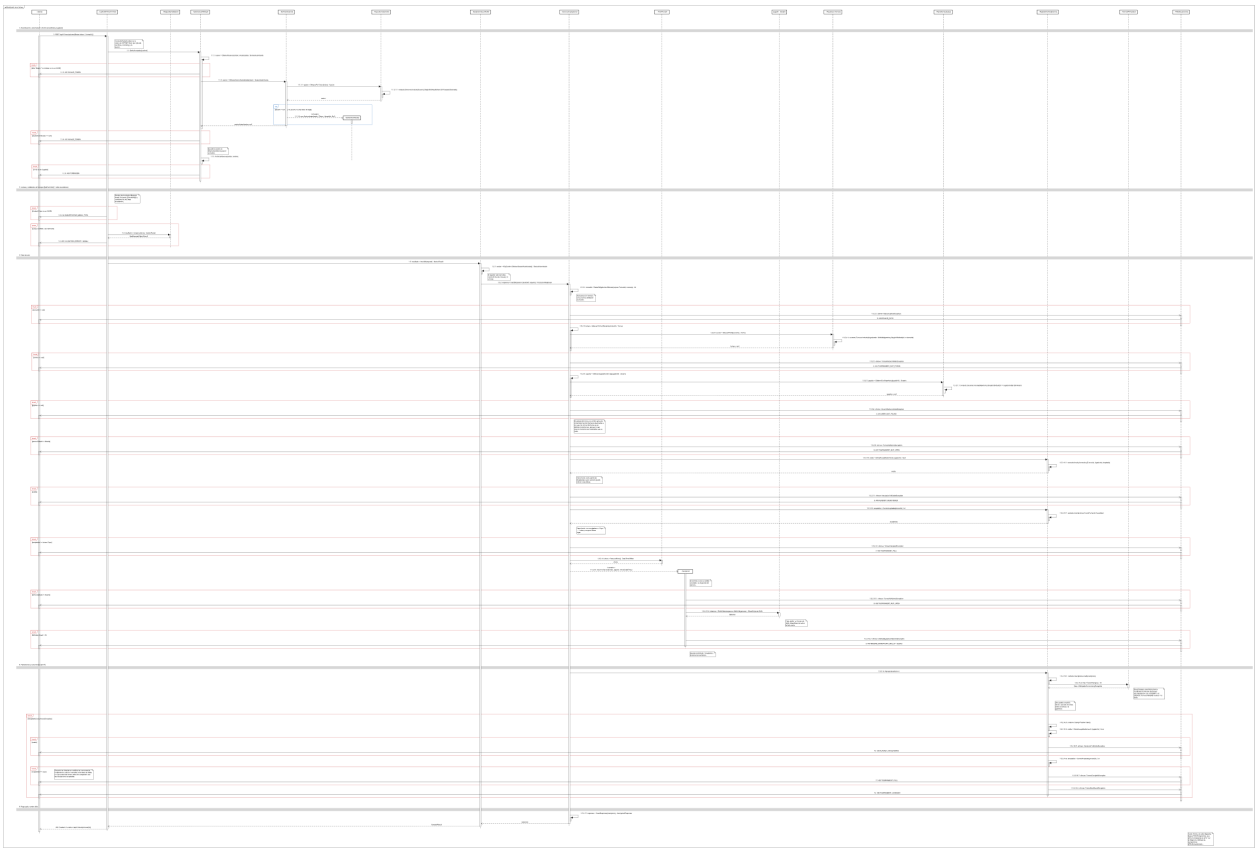
Anexo C

Modelo de tablas de la base de datos



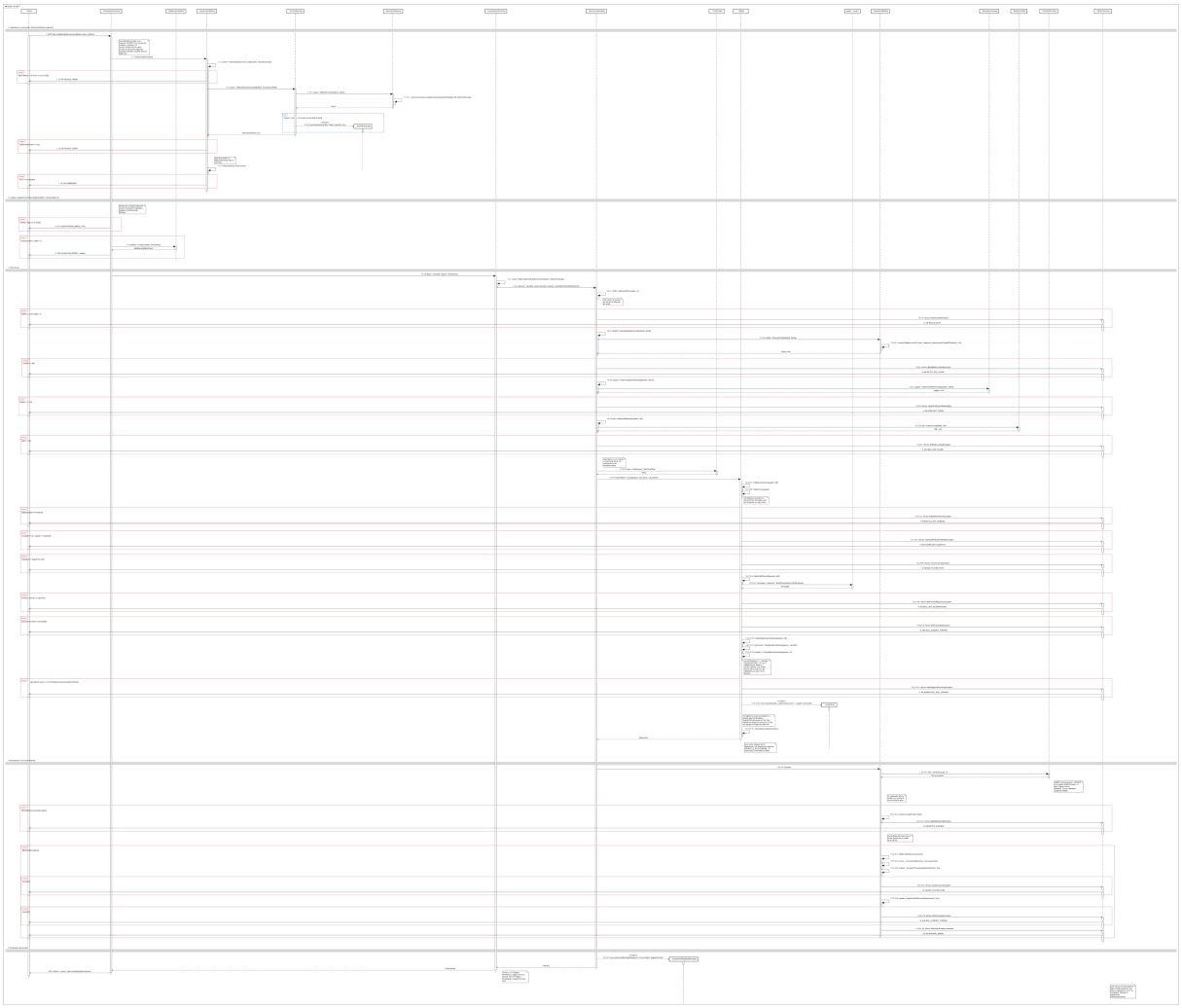
Anexo D.1

Inscripción a un torneo



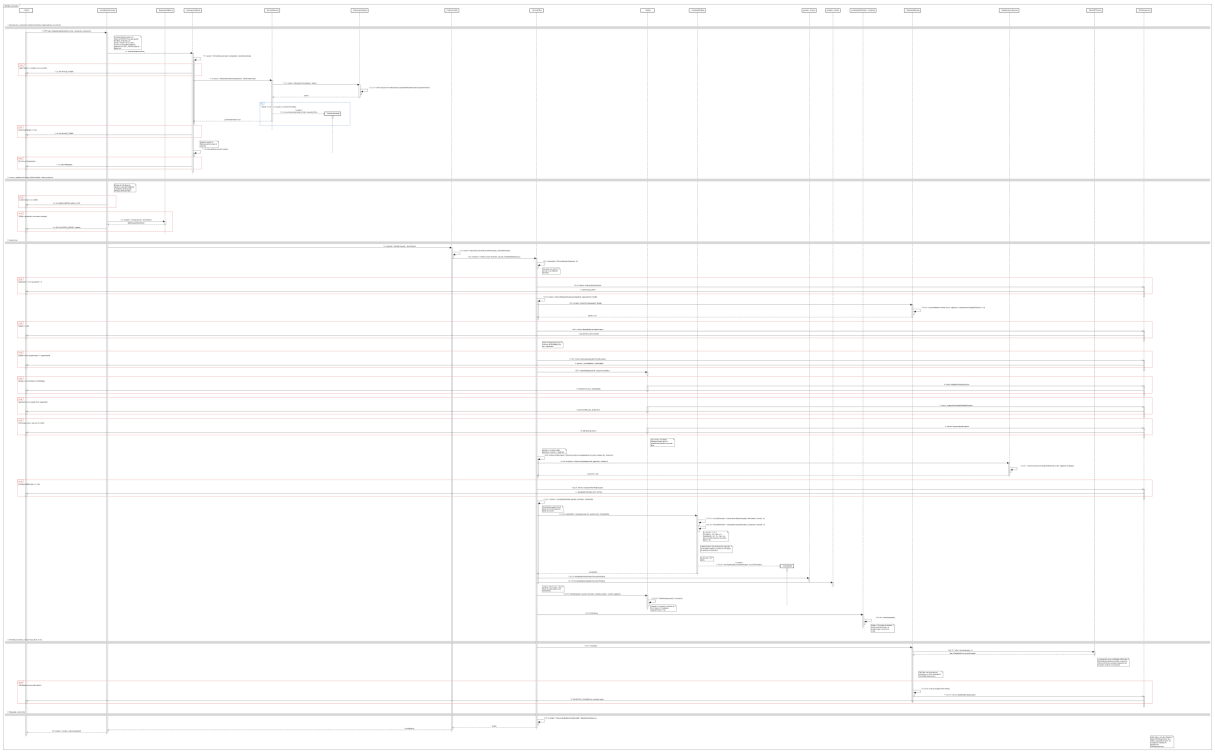
Anexo D.2

Lanzamiento de una skill



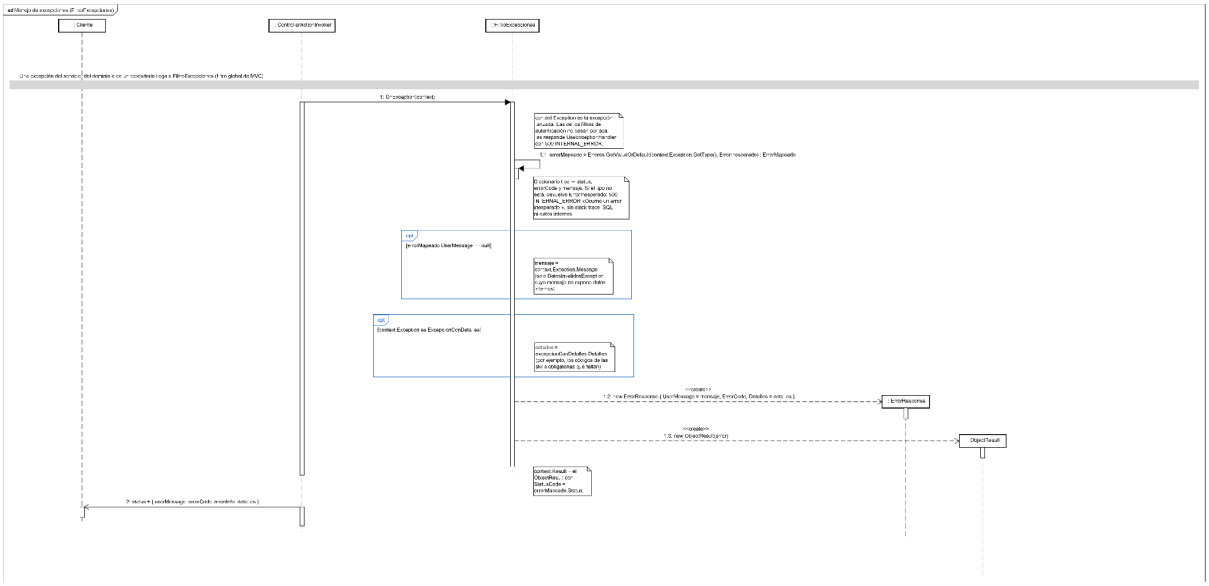
Anexo D.3

Fallo de batalla y actualización de ELO



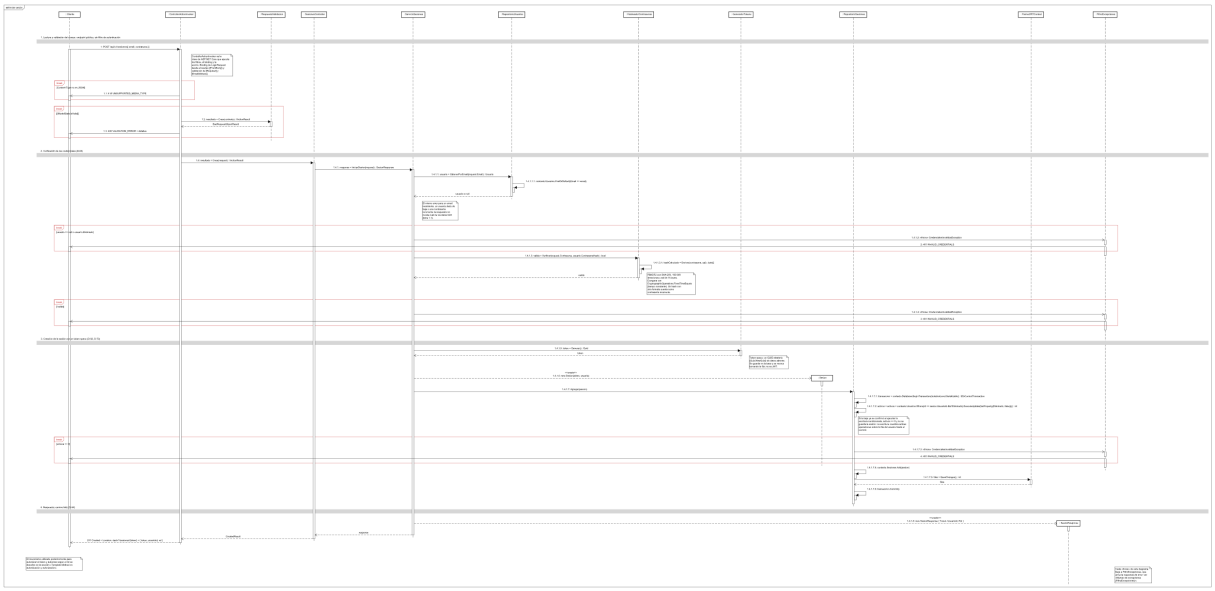
Anexo D.4

Manejo de excepciones



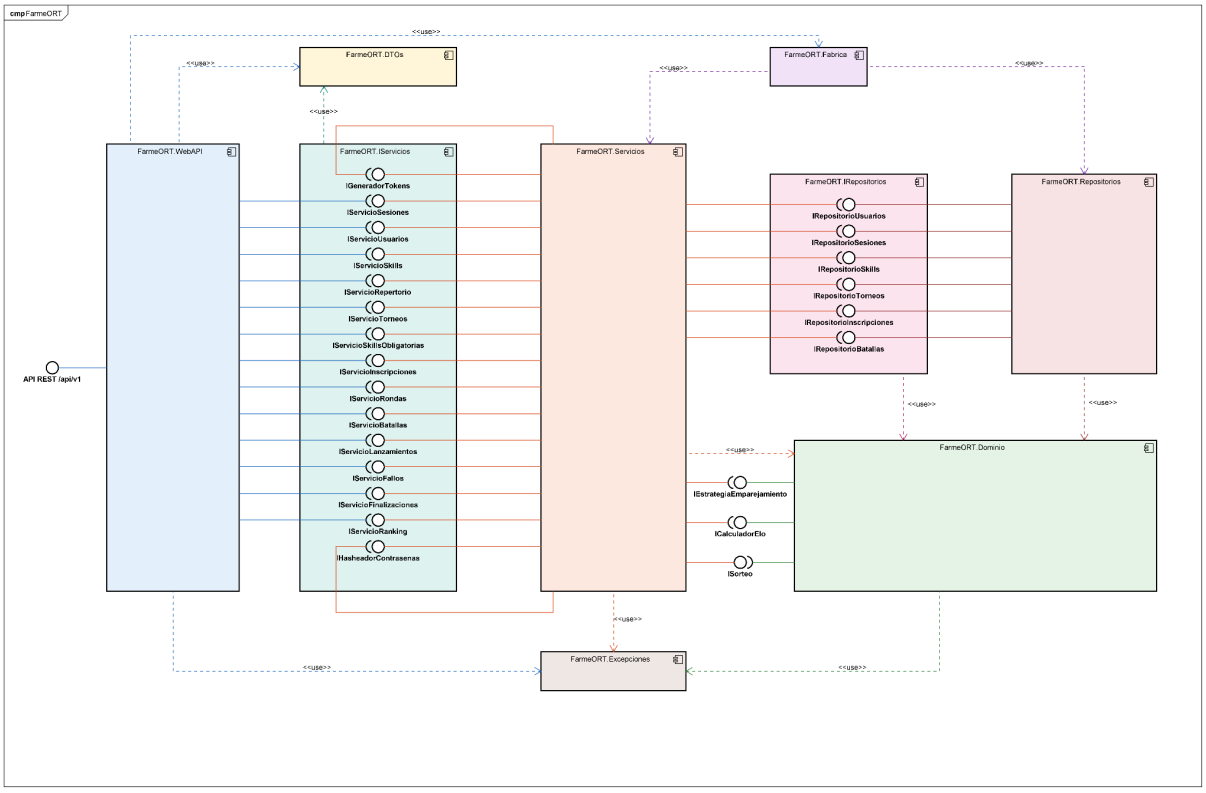
Anexo D.5

Inicio de Sesión



Anexo E

Diagrama de componentes



Anexo F

Bitácora de decisiones

F.1 — Frontera WebAPI–Servicios mediante DTOs (D-01)

Problema. Definir qué tipos atraviesan el límite entre la API REST y los casos de uso sin exponer entidades del Dominio al controller.

Alternativas. Hacer que IServicios trabaje con entidades del Dominio y mapear en WebAPI; o hacer que sus contratos reciban *Request y devuelvan *Response, dejando el mapeo en Servicios.

Decisión y justificación. IServicios utiliza DTOs. Los controllers coordinan HTTP y delegan reglas y conversiones a Servicios. Se evita acoplar WebAPI al modelo de Dominio; se acepta como contrapartida que los contratos de servicios quedan vinculados a los DTOs utilizados por la API.

Conceptos del curso. Separación de responsabilidades, bajo acoplamiento, DIP y arquitectura por capas.

Evidencia. FarmeORT.IServicios/Sesiones/IServicioSesiones.cs, FarmeORT.WebAPI/Controllers/SesionesController.cs, implementaciones en FarmeORT.Servicios y diagramas de componentes y clases.

Participación de IA y criterio humano. Se presentaron ambos esquemas y sus ventajas. El equipo eligió trabajar con DTOs, aceptando explícitamente el acoplamiento resultante al contrato de intercambio.

F.2 — Fábrica para la composición de dependencias (D-34; actualiza D-06)

Problema. El registro de implementaciones concretas desde Program.cs hacía que WebAPI referenciara las capas de Servicios y Repositorios, contrario al objetivo de independencia entre librerías.

Alternativas. Mantener el composition root en WebAPI; exponer implementaciones; o introducir un assembly encargado del registro sin hacer públicas las clases concretas.

Decisión y justificación. FarmeORT.Fabrica expone AgregarDependencias, registra TimeProvider y delega en RegistroServicios y RegistroRepositorios. Las implementaciones continúan encapsuladas como internal, y DisableTransitiveProjectReferences evita que WebAPI acceda a ellas por referencias transitivas. Así, WebAPI programa contra contratos y la fábrica conoce las implementaciones que debe conectar.

Conceptos del curso. DIP, Inversión de Control, inyección de dependencias, GRASP Indirection e independencia de librerías.

Evidencia. FarmeORT.Fabrica/RegistroDependencias.cs, FarmeORT.WebAPI/Program.cs, FarmeORT.WebAPI.csproj, pruebas de RegistroDependencias y diagrama de componentes.

Participación de IA y criterio humano. Claude Code propuso la fábrica al contrastar el diseño con observaciones docentes. El equipo adoptó esa opción y, tras detectar que las referencias transitivas todavía exponían implementaciones, incorporó una restricción adicional; se descartó conservar el acoplamiento original.

F.3 — Autenticación y autorización mediante un punto de extensión (D-13; D-14)

Problema. Garantizar que ningún endpoint autorice por rol sin autenticar antes y evitar duplicar la lectura y validación del token.

Alternativas. Usar dos filtros independientes con orden preestablecido o hacer que el filtro de autorización especialice el de autenticación.

Decisión y justificación. AutorizacionAttribute hereda de AutenticacionAttribute y sobrescribe AutorizarSesion. El filtro base fija el flujo de validación y deja el control específico del rol a la subclase. Los filtros producen directamente los errores previstos 401 y 403 con ErrorResponse, pues esos rechazos no se traducen mediante el filtro global de excepciones.

Conceptos del curso. Template Method, SRP, OCP, reutilización y separación entre autenticación y autorización.

Evidencia. FarmeORT.WebAPI/Filtros/AutenticacionAttribute.cs, AutorizacionAttribute.cs, pruebas de ambos filtros y diagrama de clases de seguridad.

Participación de IA y criterio humano. La IA comparó independencia frente a herencia; el equipo seleccionó la segunda por reducir duplicación y garantizar el orden, manteniendo en Servicios los permisos específicos sobre recursos.

F.4 — Reglas de batalla en la entidad y sorteo inyectable (D-51)

Problema. El registro de turnos, la repetición de skills y el cambio de estado de una batalla debían comportarse igual sin importar qué caso de uso modificara la batalla; además, el desempate inicial por azar debía poder probarse de forma repetible.

Alternativas. Implementar reglas y sorteo en el servicio, usar una implementación aleatoria directamente desde la entidad, o encapsular las reglas en Batalla e inyectar una abstracción de sorteo al crearla.

Decisión y justificación. Batalla valida sus invariantes y controla turnos, lanzamientos y estados. Recibe ISorteo solo en la creación; SorteoAleatorio se ubica en Servicios. Así, el azar es reemplazable en pruebas y las reglas permanecen junto a los datos que necesitan.

Conceptos del curso. GRASP Information Expert y Creator, DIP, Protected Variations, encapsulamiento y TDD.

Evidencia. FarmeORT.Dominio/Batallas/Batalla.cs, FarmeORT.Servicios/Batallas/SorteoAleatorio.cs, BatallaTests, SorteoAleatorioTests y secuencia de lanzamiento.

Participación de IA y criterio humano. Codex asistió en la implementación por ciclos TDD; el equipo mantuvo el azar inyectable y descartó trasladar las reglas a Servicios o agregar una jerarquía State innecesaria.

F.5 — Cálculo de ELO reemplazable mediante Strategy (D-60)

Problema. La fórmula de ELO podía cambiar sin obligar a modificar el caso de uso que falla una batalla; ambos nuevos puntajes deben calcularse desde los valores anteriores al fallo.

Alternativas. Fórmula privada en el servicio; Template Method con una clase base; o una estrategia intercambiable inyectada.

Decisión y justificación. ICalculadorElo.Calcular recibe los puntajes previos y devuelve juntos los nuevos valores en ResultadoElo. CalculadorEloBase implementa la política actual y se registra mediante DI. Se puede agregar otra política sin cambiar ServicioFallos y probar la regla por separado.

Conceptos del curso. Strategy, OCP, DIP, Protected Variations y polimorfismo.

Evidencia. FarmeORT.Dominio/Elo/ICalculadorElo.cs, CalculadorEloBase.cs, CalculadorEloBaseTests y FarmeORT.Servicios/RegistroServicios.cs.

Participación de IA y criterio humano. Claude Code examinó alternativas y ayudó a preparar casos de prueba que distinguen errores de redondeo o de actualización secuencial. El equipo adoptó Strategy y descartó ligar el algoritmo al servicio o a una clase base extensible por herencia.

F.6 — Rondas derivadas y emparejamiento por Strategy (D-62)

Problema. Evitar duplicar el número de ronda en otra entidad y asegurar emparejamientos predecibles, independientes del orden casual de los resultados de la base.

Alternativas. Crear una tabla y entidad Ronda o derivar cada ronda de Batalla.NumeroRonda; emparejar mediante un método privado o una estrategia; cruzar posiciones adyacentes o mejor contra peor.

Decisión y justificación. Las rondas son agrupaciones de batallas por número, sin tabla independiente. IEstrategiaEmparejamiento abstrae el algoritmo y EmparejamientoPorElo ordena por ELO descendente, desempata por id y cruza mejor contra peor. ServicioRondas prepara la ronda, mientras GuardarRonda confirma de forma atómica las batallas y el cambio del torneo.

Conceptos del curso. Strategy, Information Expert, OCP, DIP, consultas deterministas y atomicidad.

Evidencia. FarmeORT.Dominio/Rondas/EmparejamientoPorElo.cs, ServicioRondas.cs, RepositorioBatallas.GuardarRonda, pruebas de emparejamiento y rondas, diagrama de clases.

Participación de IA y criterio humano. Claude Code propuso las alternativas; el equipo eligió ronda derivada y mejor contra peor. Las revisiones posteriores exigieron hacer explícita la atomicidad y rechazar estados inconsistentes; la solución fue ajustada y probada.

F.7 — Inscripción como agregado de reglas de elegibilidad (D-41; actualizado por D-67)

Problema. La inscripción reúne condiciones de torneo, jugador, cupo, duplicados y skills obligatorias. Era necesario distribuir estas validaciones sin convertir al repositorio en responsable de reglas de negocio.

Alternativas. Resolver todas las reglas en Servicios; ubicar en repositorios consultas que también decidieran negocio; o dejar invariantes propias en Inscripcion y coordinar las consultas entre agregados en el servicio.

Decisión y justificación. ServicioInscripciones consulta datos y controla el estado Abierto antes de duplicado y cupo (precisión posterior D-67). Inscripcion protege sus invariantes en el constructor. Las excepciones de skills faltantes incluyen códigos identificables en detalles sin alterar el formato general de ErrorResponse.

Conceptos del curso. Information Expert, Creator, SRP, coherencia de contratos REST y TDD.

Evidencia. FarmeORT.Dominio/Inscripciones/Inscripcion.cs, ServicioInscripciones.cs, pruebas correspondientes, FiltroExcepciones y diagrama de secuencia de inscripción.

Participación de IA y criterio humano. Claude Code propuso alternativas; el equipo aprobó concentrar invariantes en Dominio. Una auditoría posterior detectó que el orden de controles devolvía un motivo de rechazo engañoso, por lo que se corrigió con RED/GREEN en D-67.

F.8 — Cambios multientidad con un solo guardado (D-42 y D-54)

Problema. Cancelar un torneo afecta torneo, batallas e inscripciones; fallar una batalla afecta resultado, ELO de dos jugadores y eliminación de la inscripción del perdedor. Guardar cada elemento por separado podría dejar el sistema en un estado parcial.

Alternativas. Varios SaveChanges con transacción y abstracción nuevas; actualizaciones SQL directas; o entidades rastreadas bajo un mismo FarmeORTContext Scoped con un único SaveChanges.

Decisión y justificación. Cada servicio verifica precondiciones, modifica las entidades mediante sus métodos de dominio y realiza un solo guardado. EF Core persiste o revierte conjuntamente los cambios. En el fallo, además se controla la concurrencia del estado de Batalla y del ELO. No se introdujo una Unit of Work nueva.

Conceptos del curso. Atomicidad, transacciones, DI Scoped, SRP, Information Expert y concurrencia optimista.

Evidencia. ServicioTorneos.Cancelar, ServicioFallos.Fallar, RepositorioTorneos.Actualizar, RepositorioBatallas.Guardar, FarmeORTContext y pruebas de rollback y concurrencia.

Participación de IA y criterio humano. Claude Code y Codex compararon alternativas en los dos casos. El equipo descartó la transacción explícita con otra abstracción; los tests detectaron que el ELO necesitaba protección adicional frente a modificaciones concurrentes y se agregó el token correspondiente.

F.9 — Coordinación de inscripciones, rondas y fallos concurrentes (D-70, D-71 y D-74)

Problema. Dos pedidos podían confirmar inscripciones por encima del cupo, generar dos rondas iguales o preparar una ronda con jugadores que otra solicitud ya había eliminado o retirado.

Alternativas. Volver a comprobar condiciones antes de guardar, locks en servicios Scoped, aislamiento más fuerte con relecturas, restricciones independientes o una versión de concurrencia compartida en el torneo.

Decisión y justificación. Se agregó Version como propiedad shadow de concurrencia en Torneo. Las escrituras relevantes renuevan la versión dentro del mismo guardado: nuevas inscripciones, retiro individual, generación de rondas y eliminación por fallo. Un intento con versión desactualizada se revierte; el cliente recibe un 409 y debe volver a consultar, sin reintento automático. La solución preserva la separación entre reglas de Dominio y coordinación de persistencia.

Conceptos del curso. Concurrencia optimista, transacciones EF Core, TDD de regresión, DI Scoped y consistencia de estado.

Evidencia. FarmeORTContext.SaveChanges, RepositorioInscripciones.Actualizar, RepositorioBatallas.GuardarRonda, ServicioRondas, RepositorioInscripcionesTests y RepositorioRondasTests con contextos independientes.

Participación de IA y criterio humano. Codex ayudó a reproducir carreras y comparar soluciones. Se descartaron locks Scoped y relecturas aisladas porque no cerraban la ventana de concurrencia. Nuevas reproducciones (D-71 y D-74) llevaron a ampliar el uso de Version, en lugar de afirmar que D-70 ya cubría todos los casos.

F.10 — Formato uniforme y seguro de errores HTTP (D-07, D-10 y D-65)

Problema. Los errores podían provenir de excepciones de negocio, validación MVC, autenticación o fallos fuera del pipeline de controllers, y producir cuerpos heterogéneos o revelar información técnica.

Alternativas. try/catch por action o filtro, respuestas predeterminadas del framework, o manejadores centralizados con un solo contrato.

Decisión y justificación. FiltroExcepciones mapea excepciones conocidas a errores HTTP; RespuestaValidacion construye el 400 del mismo formato; RegistroWebApi configura manejo para 500 y respuestas sin cuerpo de ciertos estados. Los filtros de autenticación y autorización producen sus 401/403 previstos. Un error inesperado no expone stack traces ni mensajes SQL.

Conceptos del curso. SRP, OCP, contratos REST, seguridad, encapsulamiento de errores.

Evidencia. FarmeORT.WebAPI/Filtros/FiltroExcepciones.cs, ErroresGenerales.cs, Validaciones/RespuestaValidacion.cs, RegistroWebApi.cs y sus pruebas.

Participación de IA y criterio humano. La IA propuso centralizar; el equipo adoptó una respuesta homogénea. Las pruebas de extremo a extremo mostraron que el filtro MVC solo no

cubría todas las respuestas, por lo que se amplió el manejo sin repetir try/catch en los controllers.

F.11 — Contrato REST corregido según observaciones docentes (D-69 y D-72)

Problema. Algunas rutas y decisiones previas eran inconsistentes con las observaciones docentes: recursos en singular, uso de actual como identificador y reglas del ranking o del cupo de torneos contrarias a respuestas oficiales del foro.

Alternativas. Mantener las rutas y justificar sus excepciones, agregar identificadores nuevos o reutilizar identificadores existentes; conservar decisiones previas del equipo o aplicar las aclaraciones vigentes.

Decisión y justificación. Se unificaron rutas como /rankings y /repertorios, y DELETE /sesiones/{token} identifica la sesión mediante su token. Tras el foro, el ranking muestra ganadas/perdidas globales, aunque se filtre por torneo; un cupo no puede bajar por debajo de las inscripciones aceptadas; un autenticado puede consultar el ranking de un torneo cancelado. Prevalecieron las aclaraciones oficiales sobre las decisiones anteriores.

Conceptos del curso. REST, identidad de recursos, consistencia de API, status HTTP, Information Expert y trazabilidad de requisitos.

Evidencia. RankingController, RepertorioController, SesionesController, ServicioTorneos, ServicioRanking, sus tests y especificación vigente de la API.

Participación de IA y criterio humano. Claude Code comparó las observaciones docentes con el código e identificó diferencias. El equipo eligió las rutas y aprobó las correcciones, modificando decisiones anteriores como D-04, D-38, D-45 y D-46. No se asumieron como requisitos preguntas estudiantiles sin respuesta oficial.

F.12 — Simplificación deliberada del código y pruebas sin depender de detalles internos (D-68)

Problema. Distintos estilos y pruebas acopladas a campos privados dificultaban que cualquiera de los integrantes comprendiera y explicara la solución en la defensa.

Alternativas. Conservar estilos personales, adoptar construcciones modernas por todo el repositorio, o utilizar un estilo explícito y uniforme que satisfaga analizadores y conserve comportamiento.

Decisión y justificación. Se prefirieron validaciones if expresas, métodos con nombres descriptivos, responsabilidades separadas, fixtures preparados mediante operaciones del Dominio y verificaciones de interacciones reales en los tests. Los cambios sin variación funcional se trataron como REFACTOR; los defectos detectados se corrigieron con nuevos ciclos RED/GREEN.

Conceptos del curso. Clean Code, refactoring, TDD, FIRST, SRP y encapsulamiento.

Evidencia. Commits de la rama de simplificación, ServicioUsuarios, ServicioRanking, ServicioRondas, pruebas de servicios y repositorios y verificaciones de formato y build.

Participación de IA y criterio humano. Claude Code propuso uniformar el código. El integrante delimitó el alcance, exigió preservar comportamiento y descartó propuestas que cambiarían JSON o ampliarían refactors fuera de contexto. Los tests se usaron para verificar que la simplificación no alterara los contratos.